Abstract

Anonymity is often a key talking point when presenting the concept of cryptocurrencies. This work first focuses on the building blocks of the Bitcoin cryptocurrency and its relationship with cryptocurrencies Ethereum and Dash. Then it looks at known past and present privacy pitfalls. First it focuses on on-chain anonymity and its shortcomings. Namely address reuse and common input clustering. The work then analyzes these shortcomings and their imperfect existing solutions. The work then shifts focus to off-chain anonymity. That is, the supporting services without which cryptocurrency networks could not function like cryptocurrency wallets. It takes a look at key storage, different types of cryptocurrency wallets and how they communicate with cryptocurrency networks. This is followed by the introduction of an attacker model. The attacker is an eavesdropper that can listen to, but cannot manipulate traffic between a cryptocurrency wallet user and a cryptocurrency node. Properties and capabilities of such an attacker follow. The work then takes a look at three cryptocurrency wallets: Trezor Suite, Ledger Live a Electrum Wallet on which it demonstrates the properties described above in practice. An application in designed and implemented based on all the learned knowledge. The goal of this application is to aid a user in determining, whether captured network traffic contains communication generated by a cryptocurrency wallet. This is followed by a conclusion and proposition of future work.

Abstrakt

Při diskusích o kryptoměnách je častým tématem anonymita. Tato práce se věnuje stavebním blokům kryptoměny Bitcoin a jak na ni navazují kryptoměny jako Ethereum a Dash. Dále popisuje, jak a kde se v minulosti i přítomnosti v kryptoměnových sítích ztrácí soukromí a anonymita. Prvně se zaměřuje na anonymitu, respektive její nedostatky v samotném distribuovaném bloko-řetězu. Práce popisuje efekt znovu-využívání adres a shlukovací heuristiku založenou na společných vstupech transakce. Analyzuje je a popisuje nedokonalé aktuální řešení. Dále se práce zaměřuje na anonymitu mimo bloko-řetěz. Tedy na podpůrné služby, které jsou nedílnou součástí funkce kryptoměnových sítí jako jsou kryptoměnové peněženky. Práce diskutuje úschovu klíčů, různé druhy peněženek a jak komunikují s kryptoměnovými sítěmi. Následuje návrh modelu útočníka, jež je schopen odposlouchávat komunikaci uživatele kryptoměnové peněženky. Avšak nedokáže ji měnit. Jsou popsány vlastnosti, schopnosti a očekávaná úskalí takového útočníka. Poté práce věnuje pozornost třem kryptoměnovým peněženkám: Trezor Suite, Ledger Live a Electrum Wallet. Na těchto peněženkách demonstruje výše zmiňované vlastnosti kryptoměnových peněženek v praxi. Na základě všech těchto informací je navržena a implementována aplikace, jež je schopna napomoci uživateli určit, zda zachycený síťový provoz obsahuje komunikaci generovanou kryptoměnovou peněženkou. Následuje vyhodnocení práce a její další možné rozšíření.

Keywords

Cryptocurrency, Bitcoin, Anonymity, Deanonymization, Computer Networks, Metadata, Eavesdropping, TOR, TLS/SSL

Klíčová slova

Kryptoměna, Bitcoin, Anonymita, Deanonymizace, Počítačové Sítě, Metadata, Odposlech, TOR, TLS/SSL

Reference

HROMADA, Denis. *Deep Packet Inspection of Cryptocurrency Wallet Traffic.* Brno, 2025. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Vladimír Veselý, Ph.D.

Rozšířený abstrakt

Kryptoměny jako Bitcoin, Ethereum, Monero, Litecoin či jiné jsou čím dál více populárnějším prostředkem pro digitální přenos hodnoty. Ruku v ruce s jejich rozšířením se také zvětšuje objem jejich prostředků, které jsou spojovány s nelegálními činnostmi jako jsou podvody, krádeže, výpalné za vydírání, praní špinavých peněz, obchodování s nelegálními látkami, nelegální obchod se zbraněmi a mnoho dalších. Tato práce se ve zkratce zaobírá možnostmi deanonymizace v kryptoměnových sítích. Na problematiku deanonymizace lze nahlížet z různých hledisek - analýza informací uložených přímo v jejich veřejně dostupné distribuované databázi (např. shlukování kryptoměnových adres), analýza síťové komunikace (např. využívání zpráv vyměňovaných mezi účastníky kryptoměnové sítě), analýza pomocných provozů (např. komunikace uživatelů s kryptoměnovou sítí za pomoci kryptoměnových peněženek).

Bitcoin byl uveden na svět v roce 2009 a přinesl základy, jimiž se inspirovala řada dalších kryptoměn. Transakce Bitcoinu jsou ukládány v účetní knize, jež je volně dostupná všem účastníkům sítě. Tato účetní kniha je uložená jako všemi volně čitelný bloko-řetěz, a za pomoci předem definovaných pravidel mohou do této struktury účastníci řetězit další bloky. Síť Bitcoinu je tvořena z dobrovolných účastníků, jež přispívají k jejímu chodu. Kdokoli se do této sítě může přidat, a stejně tak ji kdykoli může opustit. Přidávání transakcí mají na starost těžaři. Ti sbírají transakce, které by chtěli uživatelé sítě vykonat. Transakce sbírají do bloků, a tyto bloky, za splnění předem definovaných podmínek, mohou těžaři přidat do bloko-řetězu. Anonymita uvnitř tohoto sdíleného, všemi čitelného, bloko-řetězu je tématem diskuse již v samotném návrhu měny. Autor, Satoshi Nakamoto, varuje proti znovu využívání stejné adresy pro přijímání více transakcí. Časem se začaly objevovat další heuristiky, jež dokáží z informací uložených v bloko-řetězu nalomit anonymitu uživatelů. Obfuskací bloko-řetězu tento problém řeší kryptoměny jako Monero a ZCash, avšak popularitou Bitcoin dosud nepřekonaly. Bitcoin takto drastické změny odmítá, čímž si zachovává zpětnou kompatibilitu. Namísto toho začaly vznikat techniky a služby, za pomoci kterých je možné vzniklé heuristiky přechytračit, jako je spojování transakcí více uživatelů do jedné.

Pro interakci běžných uživatelů s kryptoměnovými sítěmi vznikly takzvané kryptoměnové peněženky. Ty se starají o ukládání informací potřebných k utracení kryptoměny a zároveň slouží jako rozhraní pro komunikaci s kryptoměnovými sítěmi. Kryptoměnové peněženky potřebují kontaktovat kryptoměnovou síť pro kontrolu zůstatku uživatele, nebo pro zaslání nové transakce. Obojí jsou citlivé operace, které mají potenciál nést zranitelná data. Pro anonymnější kontrolu zůstatku implementuje Bitcoin pravděpodobnostní filtry, za pomoci kterých tázající tázanému neodhalí, na které adresy se přesně ptá. Příjem kryptoměny naopak nevyžaduje od příjemce aktivní připojení do sítě. Odesílateli musí příjemce pouze v nějakou chvíli předat adresu, na kterou chce obnos přijmout. Při připojování do kryptoměnové sítě si musí peněženky vybrat s jakým účastníkem sítě budou komunikovat. Tato informace bývá předem daná vývojářem dané peněženky, avšak u některých peněženek je možné ji změnit.

Máme-li útočníka, který je schopen odposlouchávat komunikaci potenciálního uživatele kryptoměnové peněženky, informace jako tyto, mohou být využity k identifikaci provozu peněženky, případně i jejího typu.

Tato práce se zabývá tímto modelem útočníka do hloubky. Popisuje jeho vlastnosti a schopnosti. Analyzuje, co přesně obsahuje síťový provoz tří existujících kryptoměnových peněženek (Trezor Suite, Ledger Live a Electrum Wallet). A na základě všech těchto

informací navrhuje a implementuje aplikaci, jež je schopna napomoci uživateli určit, zda zachycený síťový provoz obsahuje komunikaci generovanou kryptoměnovou peněženkou.

Nakonec je implementovaná aplikace podrobena testům. Výstupem těchto testů je, že za pomoci implementované aplikace je možné určit, jaká kryptoměnová peněženka byla použita. A to za předpokladu, že zachycený provoz obsahuje komunikaci, jež peněženky generují při jejich prvotním zapnutí. Následuje vyhodnocení práce a její další možné rozšíření.

Deep Packet Inspection of Cryptocurrency Wallet Traffic

Declaration

I hereby declare that this Bachelor's thesis was based upon the authors' previous original work[13]. I declare that work was prepared with the assistance of large language models, and as an original work by the author under the supervision of
Mr. Ing. Vladimír Veselý Ph.D.

.....
Denis Hromada
June 17, 2025

Acknowledgements

Tímto bych chtěl poděkovat za psychické a materiální zázemí celému mému blízkému okolí, bez kterého bych se studiu nemohl věnovat. Jmenovitě bych chtěl poděkovat vedoucímu mé práce Vladimírovi, za jeho věčnou trpělivost a přívětivý přístup. Stejně tak bych chtěl poděkovat svým rodičům a v neposlední řadě Ing. Janu Zavřelovi za kontruktivní kritiku a vlídný přístup. Zároveň přidávám recept na mé oblíbené jídlo, které zasytí libovolného čtenáře, stejně jako obsah této práce. Jedná se o recept na **jogurt s banánem, datli a burákovým máslem**. Nejprve nakrájíme jeden banán na kolečka a tři datle nakrájíme na malé kousíčky. Do prázdné mísy přesuneme lžící zhruba 250 gramů bílého jogurtu. Následně přesuneme nakrájený banán a datle do jogurtu a přidáme mazacím nožem trošku burákového másla. Následně vše polévkovou lžící zamícháme a můžeme konzumovat.
Dobrou chuť!

Contents

List of Figures

Chapter 1

Introduction

The goal of this work is to propose and design an application capable of breaking some factor of anonymity in cryptocurrency networks. It proposes an attacker model that could break the anonymity of the targeted users based on their network traffic. It then designs, implements, and tests an application that could be used by such an attacker.

Conversations about cryptocurrencies usually mention anonymity and Bitcoin in some way or another. Therefore, it seems that understanding past and current privacy vulnerabilities in Bitcoin is the logical starting point to detect potential weak spots in current cryptocurrency systems. As such, the work first introduces the basic concepts of cryptocurrencies, starting with Bitcoin and its inner workings. Its relationship to other cryptocurrencies such as Ethereum and Dash (see Chapter 2). This is followed by a description of on-chain privacy in Bitcoin (see Chapter 3). Starting with the history of privacy, prominent existing solutions, and their pitfalls. After that, off-chain privacy is covered (see Chapter 4). Here, the concept of cryptocurrency wallets is introduced, as well as their different types. It discusses the capabilities of commonly available wallets, their commonalities, and differences. This introduction leads into a description of network communication that a cryptocurrency wallet may and must generate in order to communicate with the cryptocurrency network.

Once all of the necessary theory is laid out, the work proposes an attacker model (Chapter 5). The model describes an eavesdropper that is capable of monitoring but not modifying the network communication of a cryptocurrency wallet user. The properties and capabilities of such an attacker are described. This is followed by an analysis of three selected cryptocurrency software wallets and their network traffic.

Based on the information collected, an application design is proposed (see Chapter 6). It describes a web application that can analyze captured network traffic to help determine whether it contains communication generated by a cryptocurrency wallet. Major design considerations and a set of technologies for the development of the application.

The implemented application is introduced in Chapter 7. The applications' packet capture analysis process is described along with a database schema used by the developed application. This is followed by Chapter 8, which details scenarios in which the application was tested.

Finally, the results of the tests are presented along with an evaluation and proposed future work.

Chapter 2

Cryptocurrency Basics

In 2009, Satoshi Nakamoto published the Bitcoin whitepaper [20], which laid out the principles used in Bitcoin. Since then, many new cryptocurrencies have tried to innovate on the foundation laid by Bitcoin. Cryptocurrencies relevant to this work are Ethereum and Dash.

2.1 Bitcoin

The central component of the Bitcoin network is a ledger that keeps track of transactions within the network. This ledger is stored in a *blockchain*. An identical copy of the blockchain is stored by the participants in the Bitcoin network called *full nodes*. Transaction data on the Bitcoin blockchain is not obfuscated and can be read by any participant. When a user wants to spend their money, they create a transaction and send it to the network. This is called an *unconfirmed transaction*. Unconfirmed transactions are shared among all nodes. The addition of transactions to the common ledger is carried out by network participants called *miners*. *Miners* listen for and collect new valid *unconfirmed transactions* into their *mempools*. *Miners* pick a set of *unconfirmed transactions* from their *mempool* and serialize them into a new block. Apart from transactions, a block also always contains the hash of the previous block on the blockchain and a nonce. To include the block in the common blockchain, the miner must provide *Proof of Work*. This is done by changing the block's nonce until the value of the resulting hash is smaller than the current target. The current target is dynamically adjusted every two weeks so that, on average, a new block is created every 10 minutes. Once the miner has created a block with a satisfying hash, they can broadcast their block to the network, and it will be accepted into the blockchain.

Miners are rewarded for their work in two ways. The first transaction in a block they create is a *coinbase transaction*, which creates new coins that the miner can collect. For Bitcoin, the amount of coins generated in a coinbase transaction started at 50 BTC and is halved every 210,000 blocks. The second way miners are rewarded is through transaction fees. The size of a block is limited to at most 4 MB, therefore, limiting the maximum number of transactions that can be included in a single block. To incentivize miners to include a transaction in a new block, users can pay a transaction fee.

2.1.1 Transactions

The transfer of cryptocurrency from one entity to another can be arranged on-chain and off-chain. The former requires a connection to the cryptocurrency network in question. The latter can be achieved by sharing a spending secret by any means possible. Off-chain cryptocurrency exchange could be considered a novelty and has a series of security concerns. As such, going forward, this work considers only on-chain means of cryptocurrency exchange.

The Bitcoin ledger does not have 'accounts' per se. Transactions operate with *Unspent Transaction Outputs* (UTXO's from now on). Transactions consist of a list of UTXO's the transaction will spend, as well as a list of UTXO's it will create, where the sum of values of all inputs is greater or at most equal to the sum of values of all created outputs. Only *coinbase transactions* can create new UTXO's without spending sufficiently valued inputs. The difference between inputs and outputs is called a *transaction fee* and is collected by the *miner* that successfully includes the transaction on the blockchain. If a user wants to encourage miners to include their transaction in the blockchain, they can raise the *transaction fee*.

Every UTXO created by a transaction is given a value and a predicate that must be fulfilled in order for the UTXO to be spent. The solution to this predicate is also called the *spending secret*. Spending UTXO's is done by specifying which UTXO is to be spent and data that satisfy the spending condition specified for said UTXO.

2.1.2 Addresses

When discussing Bitcoin transfers at a currency user level, phrases like 'send funds to my address' are commonplace. While a definition of an 'address' is absent from the original whitepaper, the first published client by Nakamoto implements what is now referred to as a legacy, version 1 or P2PKH address [21, 38]. Bitcoin addresses utilize the fact, that although the spending condition for UTXO's can be customized, there is only a handful of commonly used types of spending scripts. These tend to utilize asymmetric cryptography and cryptographic signatures to ensure only the intended recipient can unlock the created UTXO's. What is referred to as an address is simply a string deterministically derived from the type and contents of a spending script.

2.1.3 Sending Bitcoin

For the purposes of this work, the creation of a Bitcoin transaction can be divided into two steps. In the first phase, the inputs and outputs of the transaction are established. That is, the sum of the values of all inputs can be calculated and the values and spending conditions of created outputs can be established. Second, the spending condition of each UTXO is satisfied for the generated transaction. For example, if a UTXO's spending condition is one of the commonly used spending scripts utilizing asymmetric cryptography and signatures, the user generates and provides a signature of the transaction established in the first step using their private key. [20]

2.2 Other Cryptocurrencies

As of writing, it is estimated, that there are more than 10,000 cryptocurrencies [31]. Cryptocurrencies other than Bitcoin are often referred to as 'alt-coins'. While Bitcoin

is not the only major cryptocurrency by all measures, as of writing, when comparing market cap, Bitcoin is more than four times as large as the second biggest cryptocurrency – Ethereum, which itself is more than twice as large than the next largest alt-coin [30]. As such, the majority of this work will focus on Bitcoin itself, and other cryptocurrencies will be mentioned where relevant.

2.2.1 Ethereum

Ethereum, first conceived by in 2013 by Vitalik Buterin, takes a substantially different approach to digital money. It utilizes and expands the combination of a blockchain and decentralized consensus which was first introduced with Bitcoin. Seeing Bitcoin as essentially a State Transition System, where UTXO's are the state and blocks are transitions, Ethereum introduces the Ethereum Virtual Machine (EVM) along with a Turing-complete programming language aiming to create a foundation for decentralized applications.

The Bitcoin spending script is a simple stack-based programming language without loops, Ethereum identifies and addresses shortcomings of this approach such as the lack of Turing-completeness, as well as value blindness, lack of a more complicated state apart from spent and unspent TXO's and blockchain-blindness. Major notable differences include the fact that Ethereum has deflationary measures but does not have a strict supply limit. Instead of Bitcoin's UTXO model, Ethereum's blockchain contains accounts, each with its own balance.

Incoming transactions raise the balance of the destination account and outgoing transactions decrease that balance. When compared to Bitcoin's need to always spend entire UTXO's, this makes it easier to create transactions with arbitrary values because there is no need to handle change. Accounts also bring another important difference: Ethereum transactions consist of only one sender address, a single destination address, and a single value. [3]

2.2.2 Dash

While as of writing the Dash cryptocurrency is not in the top 100 cryptocurrencies when evaluated by market cap [30], Dash introduced important on-chain anonymity concepts, that were later used to improve Bitcoin.

Launched in 2014, the cryptocurrency now known as Dash modifies and builds on top of Bitcoin with the aim of fixing its shortcomings in anonymity and transaction speed. To realize its goals, Dash introduces the Dash Masternode network. Another layer of a network of nodes incentivized to serve the network with a portion of every mined block. Also Dash's reference client implements a modified and extended version of CoinJoin (described in detail in section 3.3) called DarkSend with the goal of decreasing the effectivity of on-chain deanonymization techniques known at the time and, therefore, increasing on-chain anonymity. [7] DarkSend implements the following noteworthy modifications and restrictions in its CoinJoin protocol:

- transaction inputs are limited to predetermined denominations;
- a relay system is used to conceal the client identity from the orchestrating master node.

Chapter 3

On-Chain Privacy in Bitcoin

Bitcoin's blockchain is open for everyone to read. This allows anyone to confirm, for example, whether a new block is valid. Although the blockchain does not inherently contain personally identifiable data, it does contain a transaction history for every TXO. This allows interested parties to gather information about users. If such parties have access to relevant off-chain data, such as customer records of a cryptocurrency exchange, they may be able to reveal a user's identity.

Although full anonymity is not guaranteed when working with Bitcoin, there are factors that users can affect that can improve or diminish it.

3.1 Address Reuse

Address reuse is a user pattern where two or more TXO's in the blockchain share an address. This used to be more common in early implementations until concerns over the security of this user pattern became more widespread. A key turning point was the introduction of BIP 32, Hierarchical Deterministic Wallets, which further facilitates the use of new addresses for each transaction [35].

The security concerns are as follows. Being able to spend two TXO's with the same address implies a common owner, therefore reducing the user's anonymity as well as the anonymity of their transaction counterparts. For this reason, address reuse was discouraged even in the original Bitcoin whitepaper [20]. It was also discovered that the reuse of addresses can make the private key vulnerable. Specifically, an attacker is capable of inferring the private key when the signature nonce is reused across two or more signatures generated with the same private key [27].

3.2 Common Input Owner Heuristic

The simplest way for a user to spend their cryptocurrency is to create their own new transaction, sign it, and send it out into the network. Transactions created this way are vulnerable to the *Common Input Owner Heuristic* 3.2. As the name suggests, it assumes that the inputs of a transaction belong to the same user. This can be used to *cluster* TXO's, substantially decreasing anonymity [17].

Although very powerful, the initial condition of the heuristic is very optimistic. It can be misdirected by a single multi-user transaction. This results in their respective clusters being merged into one. To partially mitigate this issue, the heuristic can be disabled

for transactions that resemble known multi-user spending schemas later described in this chapter (see 3.3).

To illustrate an example:

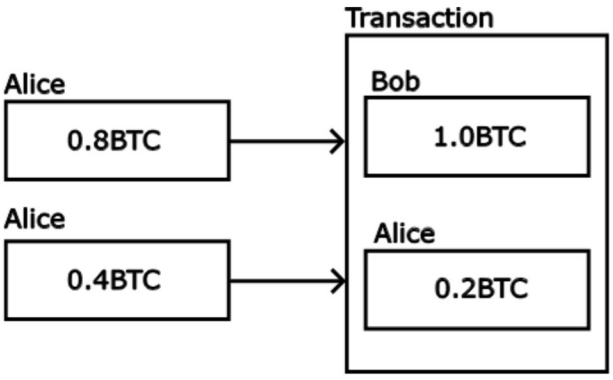


Figure 3.1: Common Input Owner Heuristic

Alice wants to send 1 BTC to Bob. Bob gives Alice a receiving address. Alice does not have exactly 1 BTC among her UTXO's, but has 0.4 BTC and 0.8 BTC. With these inputs, Alice creates a transaction where 1 BTC is sent to Bob's address, and the remaining 0.2 BTC change is sent to a new address generated by Alice (also known as a change address). She then signs the transaction for both input UTXO's and sends it into the network. The resulting transaction is shown in Figure 3.1.

An outsider Oscar, using the Common Input Owner Heuristic, will infer that both the 0.4 BTC and 0.8 BTC UTXO's were owned by a single common owner. Depending on the nature of transactions preceding this transaction, Oscar might be able to create a cluster containing a sizable subset of Alice's TXO's therefore gaining access to her transaction history.

The effectiveness of the Common Input Heuristic is improved if Alice reuses addresses or if Oscar is able to successfully identify Alice's change addresses.

3.3 CoinJoin

CoinJoin is a generalization of transactions where multiple spenders take part in the creation of a single transaction with the optional goal of better on-chain anonymity, confusing the *Common Input Owner Heuristic*.

A simple CoinJoin example would be a group of friends Alice, Bob, and Charlie who want to pay back Dave for dinner. Instead of creating one transaction per person, Alice, Bob, and Charlie can create a single transaction together. First, they tell each other what UTXOs they want to use and where potential change should go. Once they reach a consensus, they sign the transaction for each of their corresponding UTXOs and send it to the network. The resulting transaction is shown in Figure 3.2.

Although this method achieved on-chain privacy against the Common Input Owner Heuristic, the transaction participants were aware of each other's identity and their respective inputs, outputs, and spare change.

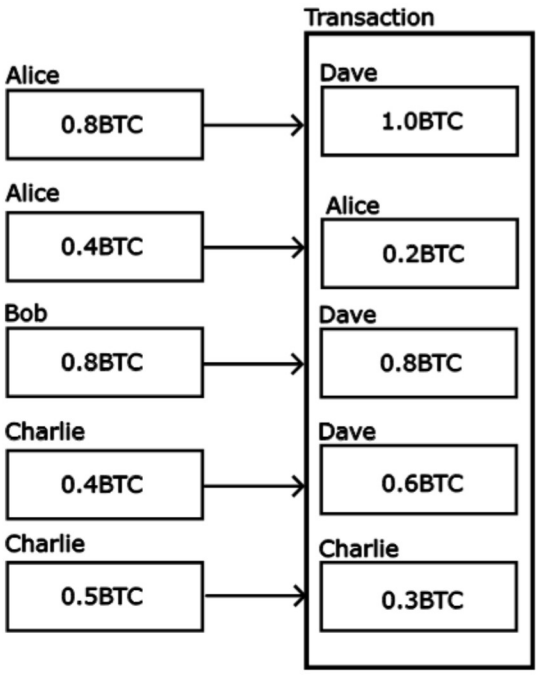


Figure 3.2: CoinJoin Illustration

Similarly, CoinJoin implementations such as JoinMarket[1] and PayJoin[2] aim to undermine the Common Input Owner Heuristic. In PayJoin, the receiving party includes one of their UTXO's as an input, blurring the line for on-chain analysis. In JoinMarket, users known as *takers* want to create a transaction with their inputs as well as inputs of another user, thus creating ambiguity in the sender-to-recipient relationships, as well as confusing the Common Input Owner Heuristic. Their counterparts, *makers*, offer their coins to be joined with for a service fee.

On the other hand, CoinJoin implementations such as Wasabi Wallet[3], and Samourai Whirpool[4] introduce a third-party middleman, known as the coordinator. The coordinator gathers groups of users looking to increase the anonymity of their coins and helps them create a joint transaction. The resulting transactions break the Common Input Owner Heuristic and aim to introduce significant ambiguity in the ownership of the created UTXO's. In both implementations, communication with the coordinator is done in a blinded, trustless manner, utilizing several Tor network identities such that even the coordinator is not able to identify which inputs are related to which outputs [9, 32].

3.4 CoinJoin Pitfalls

Although participating in CoinJoin transactions increases the anonymity of mixed coins, it does not protect users from creating transactions that undermine the achieved privacy. The Dash cryptocurrency whitepaper mentions two such cases: *Forward Change Linking*

[1] https://github.com/JoinMarket-Org/joinmarket-clientserver
[2] https://en.bitcoin.it/wiki/PayJoin
[3] https://wasabiwallet.io/
[4] https://samouraiwallet.com/

and *Through Change Linking* [7]. For example, including both a 'post-mix' coin and a 'pre-mix' coin in a single transaction undermines the entire mixing process. A later empirical study focused on CoinJoin transactions in Dash (called DarkSend) [5] also proved the effectiveness of an attack named *Cluster Intersection*, where even combining multiple 'post-mix' coins in a single transaction can reveal their owner if the coins can be traced back to the same 'pre-mix' cluster.

In summary, although CoinJoin transactions are a powerful tool for increasing on-chain privacy, users must understand their purpose and limitations to maintain anonymity.

3.5 Tumblers and Mixers

Third-party custodial mixers that offer mixing as a service typically maintain their own pool of coins, such that they are able to exchange the customer's coins for coins that do not have an on-chain connection with the input. This, for example, defeats the aforementioned *Cluster Intersection Attack*. While more effective, these services require users to trust their providers. Because the services are custodial, the user first hands off the custody of their coins and has to trust the service that they will receive the agreed-upon mixed output. Finally, since the mixing service itself decides which coins are handed out, it is aware of the input-to-output links.

Chapter 4

Off-Chain Privacy

Money is a store of value and a means of exchange, the same is true for cryptocurrencies. From a user standpoint, both storage and exchange can be done via so-called 'cryptocurrency wallets'. Importantly, the transfer of cryptocurrency is done via a connection to a cryptocurrency network.

4.1 Cryptocurrency Storage

Cryptocurrency possession is often facilitated [12] using public key cryptography. Where UTXO's are tied to a public key, and are later spent by providing a valid signature generated using a private key. Since a private key is just a string of characters, users have many options when it comes to storage.

4.1.1 Custodial Wallets

Whether it is a balance at an online cryptocurrency exchange or a different online service, a connection to a third-party service over the Internet is generally required to access stored funds. Similarly to a bank account, such services have control over the stored cryptocurrency assets, therefore, their use comes with the risk of permanently losing control of the funds in case the service goes out of business [22, 23]. Using such services for storage of larger amounts over longer periods of time goes against the decentralized foundation of cryptocurrencies.

4.1.2 Non-Custodial Wallets

On the other hand, non-custodial wallets can be independent of third-party services. A paper wallet is a type of cold wallet, where, as the name suggests, the user prints the key on a piece of paper. As an alternative to relatively fragile paper, one can, of course, embroider, engrave, or otherwise physically represent their keys. An alternative to the aforementioned methods is to store the keys digitally locally. Again, with varying degrees of resilience and security. From a plain text file on a USB stick, an encrypted drive, or a specialized digital wallet.

4.1.3 Hardware Wallets

Hardware wallets are a type of non-custodial wallets. They are specialized electronic devices with the main purpose of offline key storage [16]. An important aspect of hardware wallets

is the fact that the keys never leave the device. They only provide an application interface such that when a transaction needs to be signed, it is sent to the hardware wallet, where it is signed locally [25]. This paper will focus on hardware wallets from the company Trezor and Ledger. Specifically, Trezor's Model One, Trezor's Model T and the Ledger Nano X as these are easily available to the author.

Trezor claims to be the first company to start selling cryptocurrency hardware wallets. Their Model One entered the market in 2014 and is available for purchase on their website to this day. Its successor, released in 2018, is Model T. With their latest hardware wallet called Trezor Safe 3 released in late 2023.

Ledger originally entered the market with its Ledger Nano S in 2016 which is no longer sold in favor of its replacement: Ledger Nano S Plus. Their third product is a more expensive alternative also sold to this day: the Ledger Nano X.

It should be noted that both companies claim on their respective websites that the level of security across their own products is the same. The defining features of their more expensive products seem to be focused on user experience.

Along with the hardware wallet itself, both Trezor and Ledger provide accompanying user-facing software to interact with their respective products (Ledger Live and Trezor Suite respectively), they allow users to use their hardware wallets with third-party software as well.

4.1.4 Software Wallets

Whereas hardware wallets store keys as securely as possible, in a separate device that is seldom connected to a computer or the Internet. Software wallets are less concerned with key storage [16]. While they may work with keys stored in hardware wallets, they may also store keys on the device they are installed in – a computer or a mobile phone. Their main goal is to simply provide an interface between a user and the cryptocurrency network. Using software wallets, users can send cryptocurrency and generate addresses to receive cryptocurrency.

4.2 Existing Cryptocurrency Clients

While safe key storage is an important aspect for users, this work focuses mainly on network traffic generated by individual wallets. While both Trezor and Ledger sell multiple different hardware wallet models, the generated network traffic is dependent on the software counterpart used with the hardware wallet. That is Trezor Suite and Ledger Live respectively, but can also be entirely different software if a user decides to use a different service.

4.2.1 Ledger Live

Ledger Live is a software wallet developed by Ledger SAS to accompany their hardware wallets. It is available for desktops and Android devices and is presented as a 'one-stop shop to buy crypto, grow assets, and manage NFTs' [1].

[1] https://www.ledger.com/

4.2.2 Trezor Suite

Trezor Suite is an open-source cryptocurrency wallet developed by Satoshi Labs for use alongside their hardware wallets. While Trezor Suite used to support on-chain anonymization through rounds of Wasabi CoinJoin (see chapter 4.2.5) using the zkSnacks coordinator, this feature is no longer available following the shutdown of zkSnacks's coordinator service [26]. It is noteworthy that Trezor wallets come without pre-installed firmware, and Trezor Suite is expected to be used for initial firmware installation.

4.2.3 Electrum

Electrum is an open-source cryptocurrency wallet written in Python. In terms of key storage, it generates a locally stored `json`-like file to store private keys. By default, the stored file is encrypted using a user-generated password, although this is an opt-out option when setting up a wallet. It supports multi-signature wallets and integration with a two-factor authentication service called TrustedCoin based on 2 of 3 multi-signature wallet. Finally, it also supports a 'watching only' mode, where the user provides the wallet only with addresses, and the wallet tracks their balance without the ability to spend them.

4.2.4 Bitcoin Core

Bitcoin Core is an open-source application that can be described as a full node with wallet functionality on top. This means, compared to other wallets, it is disproportionately more resource-intensive, since it stores a major portion of blockchain data locally. This eliminates the potential vulnerability of lightweight (SPV) wallets, which depend on interacting with the cryptocurrency network and are thus vulnerable to spoofing and revealing data or metadata. E.g., when a lightweight wallet wants to calculate its balance, it has to query the network, thus potentially revealing the ownership of an address or a set of addresses. While the Bitcoin protocol implements mechanisms to limit this effect, utilizing bloom filters (BIP 37) or the later introduced Golomb-Rice Coding Sets (GCS) based filters (BIP 157 and BIP 158), sending out a query at all is inherently revealing as it is sent out at a specific time, which can also involuntarily give away information. Running a local full node eliminates these vulnerabilities altogether, since any request can be calculated from the local copy of the blockchain.

On the other hand, historically, a concern for full node security was the fact that the Bitcoin peer-to-peer protocol is not encrypted and can therefore be subject to eavesdropping and timing attacks, which could lead to identification of the node from which a transaction has originated. This concern was addressed in 2019 with BIP 324 - Version 2 P2P Encrypted Transport Protocol, which introduces an optional version of the protocol utilizing opportunistic encryption [36].

A Bitcoin full node currently requires 350GB of storage to store the entire blockchain, with extra space required every month[2]. Although it is possible to enable pruning, which can significantly reduce the required storage space down to roughly 7 GB. A semi-active internet connection is also necessary to keep the local copy of the blockchain synchronised with the network.

[2]https://Bitcoin.org/en/Bitcoin-core/features/requirements#system-requirements

Full Node - Data and Pruning

Initializing a new full node, pruned or not, requires a connection to existing networks' nodes for Initial Blocks Download (IBD[3]). In this process, the node downloads all blocks in the blockchain, verifying them in the process. Although block verification is a linear process, it can be parallelized by first downloading only the block headers, which are substantially smaller. Block download and hash calculation can then be performed in parallel, speeding up the process.

Pruned nodes, as opposed to archival nodes, do not store the blockchain in its entirety, but only select portions such that they maintain their ability to verify new blocks. While the resulting storage size of a pruned node is smaller, its initialization process still requires the entire blockchain. And since pruned nodes do not store all blocks, they cannot be used for seeding new full nodes, as they.

Bitcoin Core nodes distinguish four types of data related to the blockchain in the Bitcoin system:

- raw blocks as received over the network (`blkXXX.dat`)
- undo data (`revXXX.dat`)
- the block index
- the UTXO set

Pruning is done by deleting some of the historical raw blocks and undo data. Depending on the configuration, only the blocks from last two days may be kept[4]. A pruned node is capable of participating in the network for new block retransmission[5], but its capabilities are limited for actions requiring old block data, such as finding historical transactions for a user's wallet [2].

4.2.5 Wasabi Wallet

Developed by zkSnacks, Wasabi Wallet is a privacy-oriented software wallet. In addition to regular wallet features, it offers integration with supported hardware wallets using Bitcoin-Core HWI[6]. For each wallet, a `json` file is generated with metadata about the wallet. For non-hardware wallets, sensitive fields appear to be encrypted, while for hardware wallets, these values are simply `null`.

A selling feature of Wasabi Wallet is the passive anonymization process that it offers. A user can opt into a multi-round CoinJoin, to exponentially decrease link-ability between their coins. While ZkSnack's CoinJoin coordinator was shut down in June 2024, the wallet is open source and allows users to set a different coordinator.

It is noteworthy that although Wasabi Wallet supports hardware wallets, the passive anonymization process is not available for them. This is likely due to the need for user intervention necessary for each transaction signing.

[3]https://developer.Bitcoin.org/devguide/p2p_network.html#initialblockdownload
[4]https://Bitcoincore.org/en/releases/0.11.0/
[5]https://Bitcoincore.org/en/releases/0.12.0/
[6]https://github.com/Bitcoin-core/HWI/

4.2.6 Samourai Wallet

Samourai Wallet was an Android-focused software wallet that offered inter-connectivity with Samourai Whirlpool for on-chain anonymization via CoinJoin. Like Wasabi, Samourai Wallet offered a CoinJoin coordinator service. In April 2024, the founders of Samourai Wallet were arrested and charged with money laundering and unlicensed money transmitting offenses [34]. It is noteworthy that Samourai and Wasabi were originally developed as a single project named *ZeroLink* [10]. Only once the developers arrived at a disagreement regarding the implementation did the authors decide to split their work into two separate projects. As such, there are many shared ideas

4.3 Network Communication

Communication with a cryptocurrency network is orchestrated by cryptocurrency client software which is responsible for managing network traffic with the cryptocurrency network. The client is usually implemented as a part of the software of a wallet. Similarly to online banking apps, cryptocurrency wallets serve as an interface through which users can interact with their funds. The basic capabilities of a cryptocurrency wallet are:

- manage addresses - generate, recover and backup private and public keys of cryptocurrency addresses and accounts that are vital for any manipulation with funds;
- view balance - show the user the volume of cryptocurrency in their possession;
- send funds - allow the user to transfer their funds to another cryptocurrency user;
- receive funds - provide the user with a means of receiving funds from other cryptocurrency users.

Depending on the architecture of a specific cryptocurrency, these actions can vary in complexity and the volume of necessary communication with the cryptocurrency network.

4.3.1 Software Wallet Traffic

Knowledge of the data structures used in the Bitcoin blockchain and the Bitcoin protocol is useful when analyzing the limits of anonymity in Bitcoin (see Appendix A). Specifically, which fields and values can differ from one implementation to another, as well as the different causes of communication.

For example, receiving cryptocurrency does not require an active connection at all. The sender Alice simply writes into the blockchain a transaction generating outputs spendable by Bob. Bob could have shared how he wants to receive the funds in a previous interaction with Alice. But Bob does need to query the blockchain to determine his 'balance'. Therefore, Bob needs an active connection to either get his own full copy of the blockchain or Bob needs to query a full node.

To send a simple new transaction to the network, Alice must first know which outputs she is spending. If she also knows the destination and amount, she can create the entire transaction by herself, and she can send it to the network. Between different wallet implementations, this data might be transmitted unencrypted, encapsulated via TLS/SSL, or via Tor.

In CoinJoin transactions, Alice typically connects to a coordinator multiple times using separate Tor identities to maximize her anonymity.

The first step in running a software wallet is to determine how to connect to the cryptocurrency network. The wallet must choose a node, or a group of nodes, that it trusts. This is an issue because there is no central authority and nodes can connect and disconnect from the network at any time. Therefore, candidates for peering are usually selected from a list of predefined domain names, IP addresses, or fully qualified domain names[7,8,9], through which the wallet can establish a connection to the cryptocurrency network. This list - either hardcoded into the software or dynamically managed via the domain name system (DNS) - is usually maintained by developers of a given wallet. However, wallets may allow users to reconfigure such lists. Problems can arise if these nodes are controlled by malicious actors or if the nodes stop contributing to the cryptocurrency network [37].

Information exchanged between a cryptocurrency node and wallet is sensitive to the anonymity of the wallet user. Cryptocurrencies can have built-in countermeasures in place to mitigate some of these concerns with varying levels of success [11].

[7] https://github.com/trezor/trezor-suite
[8] https://github.com/LedgerHQ/ledger-live
[9] https://github.com/spesmilo/electrum

Chapter 5

Cryptocurrency Traffic Forensics

This chapter describes and summarizes the information gathered on existing cryptocurrency solutions. It introduces an attacker model of an eavesdropper. Then it proposes the types of information the attacker is capable of gathering.

5.1 Interception Model

This chapter assumes that an eavesdropper can obtain but not manipulate the incoming and outgoing traffic of a cryptocurrency wallet user. the described model is visualized in Figure 5.1.

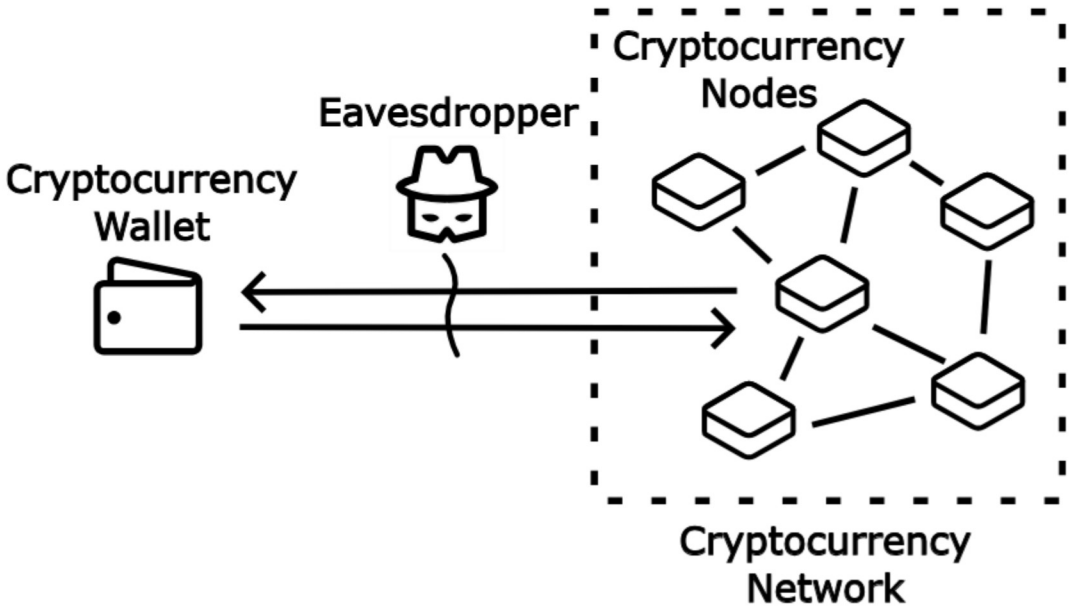


Figure 5.1: Attacker Model

5.2 Transaction Classification

Due to the difference in the number of necessary connections and participants, the proposed attacker should be capable of differentiating regular transaction broadcast

traffic from CoinJoin transaction traffic. Broadcasting a regular transaction requires only connections to a cryptocurrency node. on the other hand, the creation of a CoinJoin transaction requires several connections to an orchestrator and usually utilizes Tor.

The cryptocurrency software wallets Ledger Live, Trezor Suite and Electrum use end-to-end TLS / SSL connections for regular communication with cryptocurrency nodes. Obscuring all data exchanged in the underlying Bitcoin protocol.

For CoinJoin, Trezor Suite requires the use of its in-built Tor client, using which it presumably connects to the orchestrator. It then also disallows the user from broadcasting coins from the CoinJoin wallet without the built-in Tor client enabled [24].

On the other hand, Tor traffic in CoinJoin needs a new identity for every committed input and output. This might make it more susceptible to detection.

5.3 Wallet Provider Identification

Wallets send unencrypted DNS queries when a wallet is in use. These queries are for full node IP addresses but also for non-crucial features such as telemetry or current exchange rates. Based on these queries, it is possible to deduce which wallet type a user is using.

If unencrypted DNS queries are not present in the captured traffic, it may be possible to identify Bitcoin client server communication and the wallet provider by using existing TLS traffic classification methods:

- identifying the destination address as unique, known endpoint for a certain wallet;
- machine learning [1];
- TLS data and metadata: handshake, cipher suites, record size, packet arrival times [4, 28].

5.3.1 On-Chain Classification

It should be noted that there are currently multiple competing CoinJoin mechanisms. Previous experiments reveal [8], it is possible to reliably differentiate between them solely from on-chain information. This suggests that these competing implementations could be different enough so that it is possible to distinguish between their traffic.

5.4 Gathered Data

In order to later benchmark the resulting application, a series of network captures was taken. These captures were used to determine how the individual software wallets behave, which services they communicate with, and what kind of traffic this generates.

5.4.1 Research Traffic Samples

When researching the workings of network traffic generated by cryptocurrency wallets, network traffic captures of Bitcoin Core and the following non-full node software wallets were collected: Trezor Suite, Ledger Live, and Electrum Wallet. These captures were taken with their respective latest Windows 11 versions. Note: This is a subset of traffic samples captured in the creation of this work, which were relevant for application design.

For a description of how traffic captures were used when testing the resulting application, see Chapter 8.

For non-full node wallets, the captures contain network communication of a single wallet while performing one of the scenarios described later. For Bitcoin Core, a different set of scenarios was captured. Although the samples were captured with only one of the software wallets running at a time, without other applications running, some Windows background processes generated traffic regardless. Therefore, the traffic captured in each of the samples contains traffic from exactly one of the software wallets as well as some unrelated traffic.

These scenarios were captured for each of the non-full node wallets:

- open client - Launch the application for the first time and open its dashboard (respective `.pcapng` files are in Appendix D, `Collected Packet Captures/research/open client`);
- send transaction - With an already open client, broadcast a cryptocurrency transaction and wait until it is confirmed (Appendix D, `Collected Packet Captures/research/send-sent`);
- display balance - With an already open client, view the balance (Appendix D, `Collected Packet Captures/research/view balance`).

Because Trezor Suite supports Tor routing in its client, these tests were performed twice. Once with Tor routing enabled and once without. An additional traffic capture was also taken of toggling Tor routing on and off, respectively.

These scenarios were captured for the Bitcoin Core wallet:

- open client - launch the application for the first time Appendix D, `Collected Packet Captures/research/open client`);
- initial block download Appendix D, `Collected Packet Captures/research/initial block download`);

5.4.2 Traffic Sample Findings

Using the captured traffic, it was determined that each of the wallets investigated communicates with a different set of services. In particular, it was found that all non-full node wallets investigated use DNS to find the IP address of a cryptocurrency node to connect to. All of the investigated software wallets are open-source, and their source code is publicly available. Due to this fact, even though the captured traffic contained unrelated traffic, it was possible to cross-reference the queried domains with domains present in the software wallet source code.

For Trezor Suite and Electrum Wallet, it was also easily possible to extend the gathered list beyond domains queried in the traffic samples, as the list of used domains is stored in a handful of configuration files. on the other hand, it was found that Ledger Live's source code does not utilize configuration files to store domain names. Instead, URLs are hard-coded at every point in the code where a request is made by the application. This approach could be considered 'security through obscurity' of sorts, since it made it slightly harder to collect a list of endpoints used by the application. For the resulting list of domain names collected for each wallet, see Appendix B.

It was also found from the captured traffic that while Trezor Suite and Ledger Live wallets seem to rely largely on services available under their DNS subdomains

and are therefore likely hosted by themselves, Electrum Wallet utilizes services from various third parties. For example, Trezor Suite connects to a Bitcoin-related service under the domain entries `btcX.trezor.io` (where X is a number between 1 and 5), which is a subdomain of their product page and is therefore clearly maintained by Trezor, whereas Electrum Wallet connects to full nodes available under various domains such as `btc.ocf.sh` or `btc.electroncash.dk`, and none of the collected domains seem to be maintained by Electrum Wallet developers.

For Bitcoin Core generated network traffic, a key finding was the fact that while BIP 324 introduced an *optional* encrypted version of the peer-to-peer protocol in 2019, some network nodes are either running an outdated version of the client or deliberately communicate without encryption. So even though we were running a client that would communicate using encryption with nodes when supported. It was still forced to fall back to the unencrypted version of the protocol when the counterpart did not support the feature.

Chapter 6

Design

Based on the gathered information, research, experiments, and upon consulting the supervisor, it was decided that it would be best to design and create a network traffic analysis application that can be used to detect and identify cryptocurrency-related traffic. the application would use DNS and cryptocurrency protocol traffic to help identify which cryptocurrency product might have been used by the user. For a narrower scope, the application was to only support the Bitcoin protocol, but it should be developed so that it can be easily extended in the future. This chapter describes a set of user stories and lays out key architectural considerations when designing the application. It also introduces the technological stack for the application and finally explains how the application architecture should be structured.

6.1 User Stories

The application relies on network traffic data gathered likely through eavesdropping. As such, in the following section, the application user is referred to as Eve, and the cryptocurrency user is referred to as Alice. Eve has captured network traffic data of a suspected cryptocurrency user, Alice. When designing the application, the following use cases were considered:

- Eve suspects Alice has a cryptocurrency wallet and wants to determine its type (non-custodial or custodial). Knowledge of the wallet type would help Eve in case of further investigation. E.g., whether to look for a physical device in case of an assets search.
- Eve suspects Alice is running a cryptocurrency full node and is interested in the content of outgoing cryptocurrency protocol traffic. Such traffic is expected to contain messages broadcasting transactions and blocks. Using the proposed application, Alice should be able to browse cryptocurrency transactions and blocks broadcast from a given IP address (or IP address + port combination).
- Eve has captured network traffic and wants to determine whether it contains any signs of cryptocurrency wallet traffic. If so, the proposed application should allow Eve to identify a list of IP addresses and ports that took part in cryptocurrency-related traffic.

6.2 Key Application Design Choices

When considering designing the application, among the main considerations were the input data format, data persistency regarding the seed data, as well as the user-related input and analysis output, and finally, how the user should interact with the application.

6.2.1 Input Data Format

Network traffic analysis can be performed on the individual packet level, on flows, or on a different aggregate. It was determined that access to data inside individual packets would be beneficial for fulfilling the above-mentioned user stories. Therefore, using a data aggregate such as flows is insufficient, and the prominent packet format `.pcapng` was chosen for the application's input.

6.2.2 Data Persistency

When considering whether to implement persistent storage for the application, storage of three data categories was considered:

- seed data - E.g., DNS seeds for individual cryptocurrency wallets;
- input - packet capture (.pcapng) files;
- output - results of the application's analysis;

For seed data, it was quite apparent that it would be a better design choice to implement them in a more modular way rather than hard-coded constants. This is because the domain names used to detect and differentiate between different cryptocurrency products can be changed over time by the wallet developers. a more modular approach also allows for easier future support of more vendors and products. It was therefore decided to utilize a database for seed data storage.

For persistent input (packet capture file) storage, the ideal design choice was not as clear. a drawback of persistent file storage is larger storage space requirements, as uploaded files can be potentially very large. the benefit of storing files persistently is a simpler application flow when analyzing a file. It was also deemed a benefit to break apart the potentially time-intensive file upload step, followed by a potentially time-intensive analysis step, into two separate actions, as the absence of persistent file storage would tie these two steps into one, resulting in a potentially long cumulative waiting time. Another deciding benefit was memory requirements. Without persistent input storage, the application would need to hold the entire, potentially very large, packet capture file in memory during analysis. Given the benefits and drawbacks, persistent input storage was evaluated as a desired feature and was implemented.

The persistent data storage of analysis output was at first determined as unnecessary, as the analysis time for even a very large file was very short, and the overhead of creating database tables and filling them with analysis results was substantial. But with more complex analysis logic, the analysis time and output size grew. the deciding factor was the fact that the analysis was sometimes performing database-like queries when linking related data. Since databases are optimized for such queries, it was determined beneficial to integrate a database into the analysis flow. If a user wants to revisit an earlier analysis, the added benefit is, of course, that the computationally intensive process does not need to be performed again.

For a more in-depth overview of the final database structure, see Chapter 7.

6.2.3 Application Interface and User Flow

Given the composed user stories, the expected users of the final application are not necessarily programmers or people who are familiar with command-line tools. Therefore, it was considered better to create an application with a graphical user interface, rather than a CLI tool or an API. This allows for easier human interaction when browsing the analysis output data, as individual graphical elements can be used for navigation inside the traffic analysis output.

With the above-mentioned user stories and design choices in mind, a series of mock-ups was created to guide the development of the application's graphical interface, with the main focus on the representation of packet analysis results. The user flow proposed in the mock-ups is as follows. First, the user uploads their packet capture file to the application. a list of uploaded files can be seen in Figure 6.1. the user can then open the uploaded capture file, opening up its analysis. Upon opening a file analysis, the user is greeted with a list of endpoints (an endpoint is every unique IP address and port) with aggregate statistics for each (see Figure 6.2). Here, the user can again select one of the endpoints to open a detailed view of its network activity. the mock-up in Figure 6.3 shows a DNS-oriented view of the participant's traffic, showing if and how much traffic to individual known services was present in the analyzed file. the mock-up in Figure 6.4, on the other hand, shows a page with a table highlighting the participants' traffic over the cryptocurrency protocol.

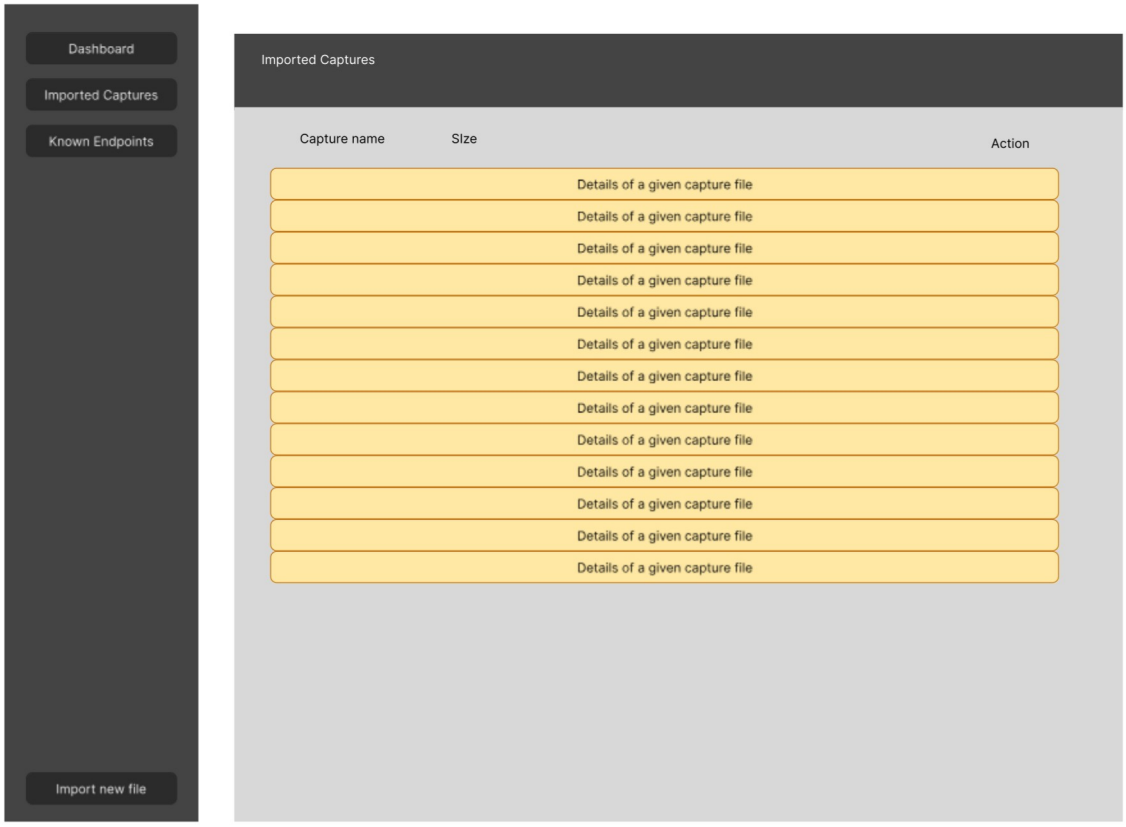


Figure 6.1: File Upload Overview Page Mock-up

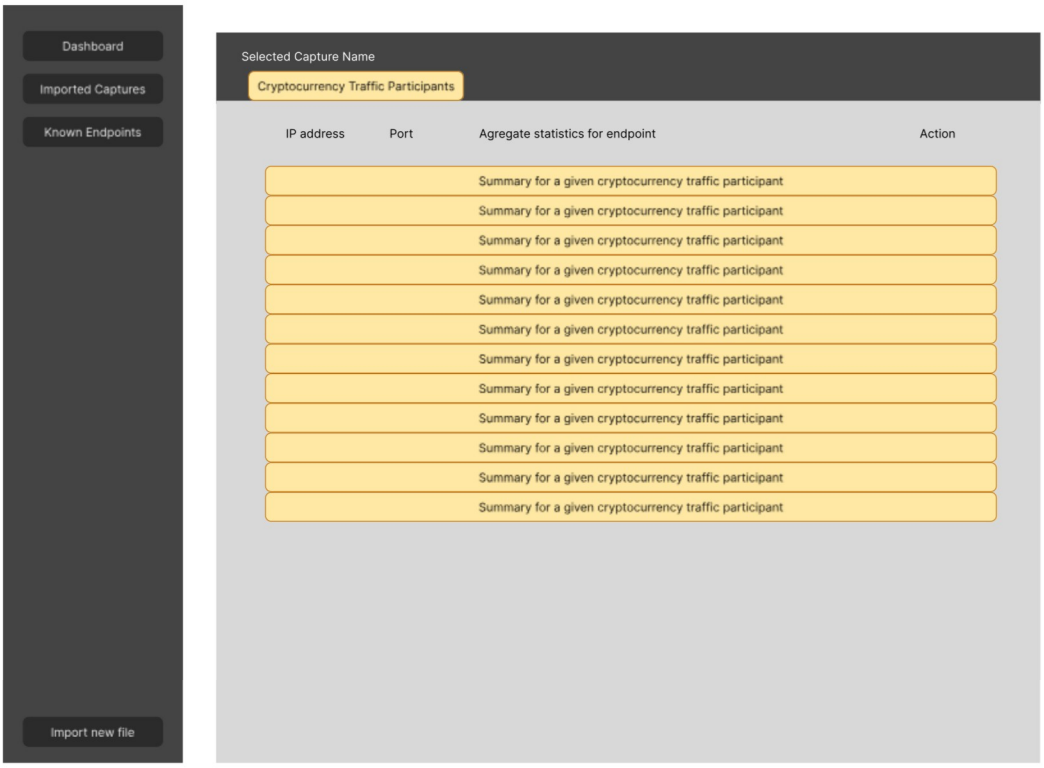


Figure 6.2: Traffic Participants Page Mock-up

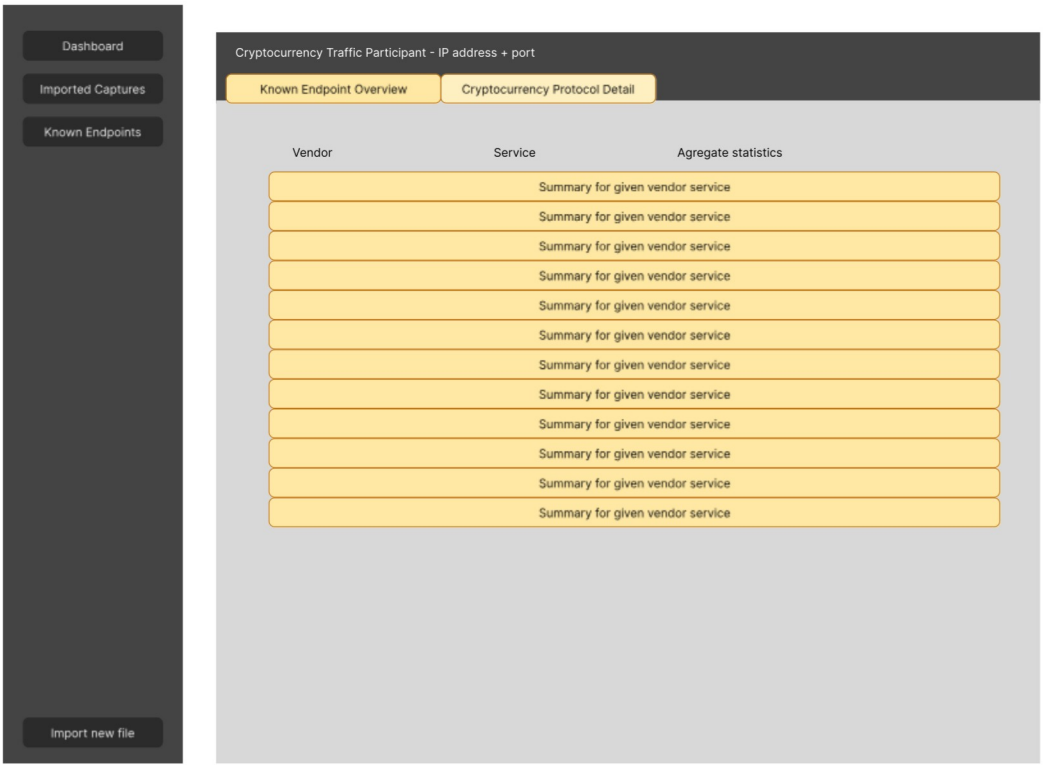


Figure 6.3: Known Endpoints Page Mock-up

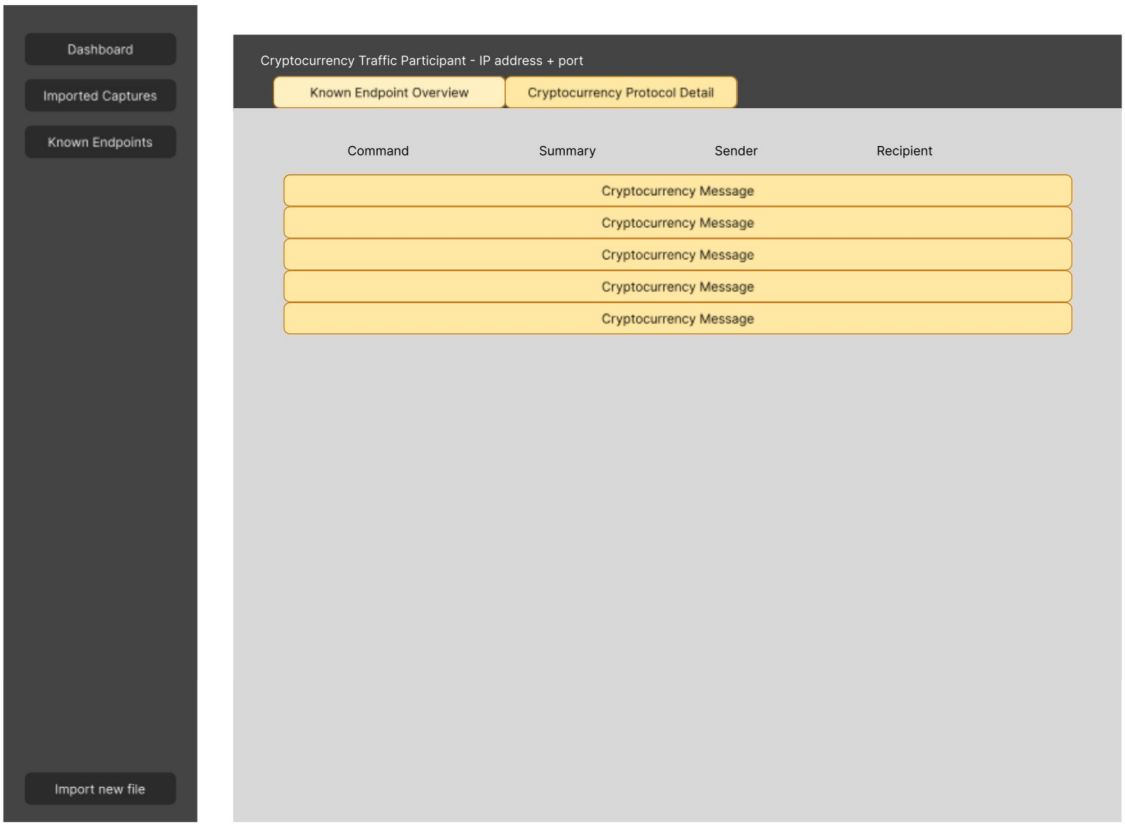


Figure 6.4: Cryptocurrency Protocol Page Mock-up

6.3 Technological Stack

When selecting a technological stack for creating the above-mentioned application, the .NET platform with Blazor was selected due to a recommendation from the advisor as well as the author's preference based on past experience with the .NET platform and other technologies. It was therefore decided that the application would be written in C# as it was the author's preference among the available alternatives, such as F# and VB.NET. Among the benefits of using the .NET platform is a large library of packages available with regard to the goals of the application. For example, basic operations the file system are handled by a part of the standard library under the `System.IO` name-space, handling of packet capture files can be delegated to the `SharpPcap` NuGet package, while database operations can be managed using `Entity Framework`. Furthermore, because Blazor is a part of the ASP.NET Core web application framework and the Blazor *server* variant was selected, it was possible to easily integrate a web interface for the created backend.

6.3.1 The .NET Platform, Blazor, and Available Libraries

The .NET platform[1] is an open-source, cross-platform software framework maintained by Microsoft. With the latest major version release[2] (.NET 9) in November 2024, it was selected as a modern, well-supported platform for building the designed application [19].

Like the .NET platform, Blazor[3] is an open-source framework developed by Microsoft. It is a part of ASP.NET Core[4], which is a cross-platform framework for building web applications. Blazor itself is a frontend framework that supports client-side, server-side rendering modes, allowing even to mix them. the implemented application utilizes Blazor's interactive server-side rendering (interactive SSR). Interactive SSR handles UI interactions over a real-time connection between the application and the browser, which, among other things, allows for easier development, avoiding the more traditional need to create API endpoints to interact with the server [19].

Libraries used by the application are managed as NuGet packages, where NuGet is a package manager for the .NET platform. The more noteworthy used libraries are PacketDotNet and SharpPcap for packet capture handling, packetnet-connections for TCP stream reconstruction, Kaitai Struct and NBitcoin for DNS and Bitcoin protocol parsing, respectively, Entity Framework for managing connections with the database, and MudBlazor, which is a component library for Blazor.

PacketDotNet[5] is a high-performance .NET assembly for dissecting and constructing network packets such as Ethernet, IP, TCP, and UDP. Whereas **SharpPcap**[6] is a .NET library for capturing packets from live and file-based devices. the implemented application utilizes SharpPcap to read packet capture files, and PacketDotNet is used to access the Ethernet, IP, and TCP or UDP contents.

To dissect DNS network protocol data structures, **Kaitai Struct** is used. Kaitai Struct is a declarative language used to describe various binary data structures, laid out in files or in memory [15]. a part of Kaitai Struct is a transpiler that translates a declaration in the Kaitai Struct language `.ksy` into a structure declaration in the desired language. Furthermore, the Kaitai Struct website also hosts an open-source 'Format Gallery' which contains user-created descriptions of binary structures. the implemented application uses a format description `dns_packet.ksy` from this 'Format Gallery' of the DNS protocol payload, transpiled to a C# class declaration. The accompanying NuGet package (KaitaiStruct.Runtime.CSharp) provides supporting C# data structures for creating and working with the generated C# Kaitai Struct structure declarations.

For working with the Bitcoin protocol, the **packetnet-connections** package is first used. It implements a TcpConnectionManager class, which can be used to track and reassemble TCP byte streams. the assembled byte streams are then handed over to Bitcoin message parsing logic. **NBitcoin**[7] is a Bitcoin library for the .NET platform, which provides low-level access to Bitcoin primitives and is used in the implemented application for parsing the reassembled TCP byte streams into individual Bitcoin network protocol messages.

[1] https://learn.microsoft.com/en-us/dotnet/core/introduction
[2] https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-9/overview
[3] https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-9.0
[4] https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-9.0
[5] https://github.com/dotpcap/packetnet/
[6] https://github.com/dotpcap/sharppcap
[7] https://github.com/MetacoSA/NBitcoin

Entity Framework Core is a .NET object/relationship mapper that enables work with a database using .NET objects [18]. the application uses Entity Framework Core to describe the database structure in a code-first approach using C# classes. This approach allows for a data structure declaration independent of the database provider.

6.3.2 Database Technology

The implemented application is designed such that it utilizes a relational database for persistent data storage. There is a number of popular relational database management systems such as Oracle, MySQL, Microsoft SQL Server, PostgreSQL, SQLite, and more [29]. Selecting which Database Management System to use was not a major concern, as they all meet the requirements of the implemented application. the latest stable PostgreSQL version (17.3) was chosen as an open-source Database Management System [33] due to the familiarity and past experiences of the author.

6.4 Architecture

The design of the application was largely inspired by what is sometimes referred to as the three-tier architecture [14]. Database access and persistent file storage operations are performed by the **Data Abstraction Layer**. the majority of application logic is contained in the **Business Layer**. That is, facades, models, analysis, and filtration logic. the web-based graphical user interface is defined in the **Presentation Layer**. Figure 6.5 shows an abstract representation of each layer and its abstracted components. the arrows represent code dependencies. For example, Facades use Unit of Work, which uses Repositories.

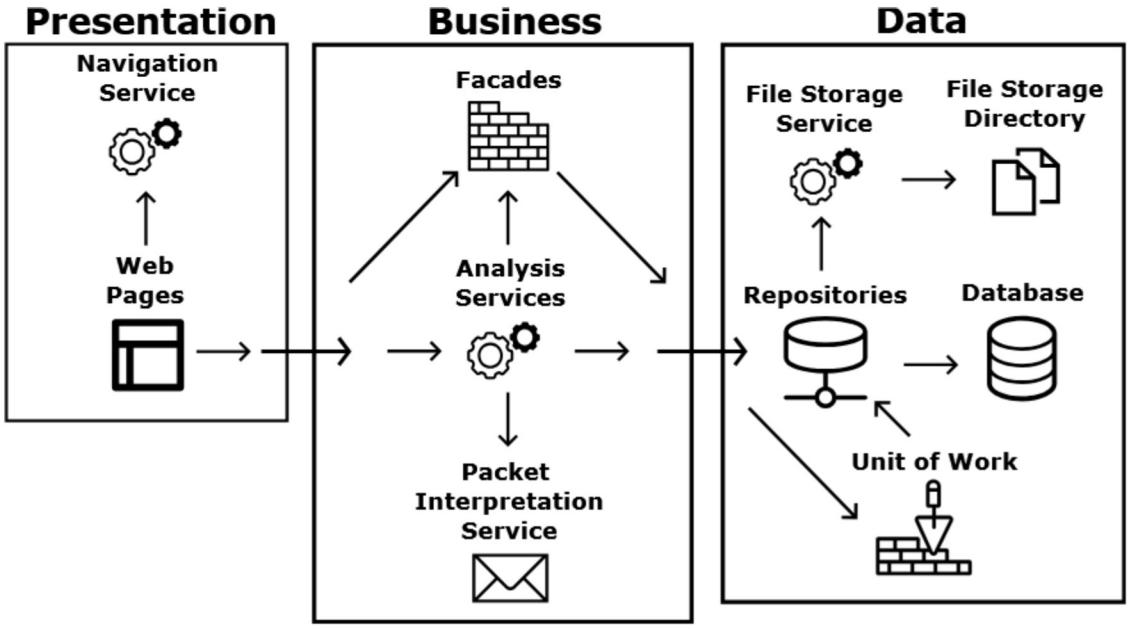


Figure 6.5: Application Architecture

Data Abstraction Layer

The data abstraction layer of the implemented application is in charge of persistent data storage. In the code, it is represented as the `Database` project. It is responsible for accessing the database and interacting with the file system. Database access is orchestrated by the Entity Framework Core NuGet package. the Data Abstraction Layer implements a *Unit of Work* abstraction over the Entity Framework database context that ensures transactional behavior when accessing the database.

Transactional behavior refers to the way operations are executed. Usually, all operations are executed as they are called. For tasks comprised of multiple operations, if an operation in the execution chain fails, the underlying data store enters an inconsistent state. Transactions implement atomicity even for task composing of multiple operations. Execute either all of the operations on success or none if one of the operations fails.

Repositories encapsulate the code-first database entity declarations while also acting as a place for extra underlying logic.

Presentation Layer

The presentation layer is responsible for the user-facing web-based interface. In the code, it is represented as the `WebApp` project. It uses the Blazor framework in combination with the MudBlazor component library. It consists of `.razor` page definitions between which navigation is handled by a custom service, that handles creation of links and navigational breadcrumbs. the presentation layer is not meant to implement heavy logic, which is delegated to the Business Layer.

Business Layer

The business layer is where the majority of the program logic is located. In the code, it is represented as both the `Business` project and the `Packet` project. This separation of concerns was deemed cleaner, as all packet parsing and interpretation is handled in the Packet project, which provides cleaner data models to the Business project, which is concerned with data manipulation rather than parsing. the Business project contains facades, models, and model mappers. Facades are an abstraction over outwardly exposed parts of the Data Abstraction Layer, most notably the Unit of Work abstraction, using which they manage read, write, and editing interactions with persistent data storage. Models are data structures outwardly used by the Presentation Layer; this is done so that internal data structures used by program logic are disconnected from the data output. the transfer of data from facades to models is performed by model mappers.

6.4.1 Containerization

Docker is a lightweight alternative to virtual machines. Instead of an entire virtual machine, Docker works with *containers*. Every container is based on an *image*. Images define the underlying blueprint for the initial container state. As with virtual machines, the provided benefit of using containers is a replicable environment to run an application. the difference from regular virtual machines is that virtual machines work with a hypervisor, whereas containers run above the operating system [6].

In application development, containers can be used to deploy applications as separate micro services, each in its own container, connected via regular computer networking.

the implemented application can be run in containers using the accompanying `Dockerfile`'s and `docker_compose.yml` files. Running the appropriate `docker compose` command (see README.md for complete instructions) launches a container with the application and a separate container with the database.

Chapter 7

Implementation

This chapter focuses on the implementation details of the created application. First, it focuses on how the application handles packet capture analysis, like which services focus on which traffic aspects and how the analysis output is persistently stored with regard to the database schema. Finally, it goes over the web interface of the application and what data individual implemented views show.

7.1 Analysis Logic

The packet capture analysis process is split into several steps. First, the EndpointAnalysisService goes through the entire packet capture file and creates a database entry for each IP address and port combination encountered, both as sender and recipient. This is so the subsequent services do not have to create sender and recipient endpoints themselves and can always rely on finding a matching ID inside the database.

Once finished, the DNS and Bitcoin analysis services are called in parallel. the DNS analysis service (DnsAnalysisService) calls a DNS-specific service in the Packet project, which reads the packet capture file, filters it for UDP packets sent from or to port 53. the DNS parsing logic uses Kaitai Struct and is explicitly looking only for a and AAAA queries and responses. It then gives a collection (IEnumerable, to be exact) of these messages back to the DnsAnalysisService. to save a DNS message in the database, the service must first find the endpoint IDs of the sender and the recipient. For DNS responses with resolved IP addresses, all endpoints with the matching IP address have to be found for inserting entries into the cross-reference table DnsMessageResolvedTrafficParticipant (See database schema in Figure 7.1). Finally, the DomainMatchAnalysisService checks whether the queried domain matches exactly or is a subdomain of a known domain, in which case an entry to the DomainMatch cross-reference table would be made. Once all DNS messages are inserted into the database, a list of all addresses resolved by DNS can be gathered. This list is then used to analyze the packet capture file one more time, this time simply to denote all traffic from and to the addresses that were resolved. This is then used to determine if a domain was only queried or if a connection to the resolved address was established as well. (See Figure 7.3).

Bitcoin traffic analysis is handled by the BitcoinAnalysisService, which calls the Bitcoin-specific BitcoinPacketAnalyzer inside the Packet project. the packet analyzer filters the traffic for the Bitcoin mainnet port 8333 and goes through the resulting

packets. Because the Bitcoin protocol is transferred over TCP, the analysis logic has to reassemble a byte stream for each IP and port sender and recipient pair. Furthermore, the captured bytestream may begin with a fragmented message that is incomplete. Every Bitcoin (mainnet) network protocol message begins with mainnet magic bytes. Therefore, the Bitcoin parser does not start interpreting messages until it locates the first Bitcoin mainnet magic. the Bitcoin message parsing logic is then handled by NBitcoin. a collection of Bitcoin messages is then returned to the BitcoinAnalysisService. to insert the message into the database, the sender and recipient are first queried from the database, so their IDs can be used in the new, inserted message model. Bitcoin headers, tx, getdata, inv, and notfound messages are implemented in the application with additional logic. Transactions and their inputs and outputs are represented as separate entities. Transactions are linked to the Bitcoin message via a cross-reference table, so that a repeated appearance of a transaction can be easily found. Similar logic is applied for block headers and inventory vector items (See database schema in Figure 7.1).

7.1.1 Database Schema

As is laid out in Section 6.2.2, it is desirable to persistently store the application's seed data, input data, and analysis output data. The application stores all data in the connected SQL database. With a slight exception for input file storage, which is managed separately, such that only file metadata and a path are stored in the SQL table, and the file itself is stored in a dedicated directory in the file system. the benefit of this approach is, files don't have to be loaded into memory in their entirety from the database when accessed. Instead, standard file system operations are performed when files are read for analysis.

Stored file metadata is contained in the `StoredFile` table, as can be seen in the Database Schema in Figure 7.1.

Known domain seed data, that is, the domains collected in Section 5.4.2. Can be imported into the application, where they are stored in the `CryptoProduct` and `KnownDomain` tables (see Figure 7.1).

The remaining tables (apart from `__EFMigrationsHistory`, which is a table created by Entity Framework Core for tracking migrations) are used for storing the analysis results. In summary, one file can have many analyses performed and stored. This is so that, perhaps, if the seeded domains change or a future change in the application is made, analysis results for the same file can be easily compared. Entities of one file analysis, such as a traffic participant (a combination of IP address and port), do not affect the same traffic participant in a different file analysis. Each file analysis tracks traffic participants and three types of *messages*. All `messages` have a sender and a recipient foreign key from the traffic participants table.

DnsMessages can be tied to known domains via a many-to-many relationship if the queried domain exactly matches or is a subdomain of a known domain. If a DnsMessage is a response that contains resolved addresses, entries in the DnsMessageResolvedTrafficPArticipant cross-reference table associate the message with participants with matching IP addresses.

Some BitcoinMessages (headers, tx, getdata, inv, and notfound) can have extra data tied to them. This is done via the BitcoinMessageInventory, BitcoinMessageHeader and BitcoinMessageTransaction cross-reference tables. the cross-reference tables are in place to remove redundancy, for example, in case a Bitcoin inventory vector item is present in multiple messages. the cross-reference tables also simplify appearance lookup

when searching for all messages where a particular entity was present. Finally, because a single bitcoin transaction can contain multiple inputs and outputs, the BitcoinTransaction table is in a one-to-many relationship with BitcoinTransactionInput and BitcoinTransactionOutput tables.

The entire database schema can be seen in Figure 7.1.

7.2 Analysis Views

When a file is uploaded to the application, the application stores it locally and allows the user to perform an analysis. (See Figure B.1.) the user can then select their file, which opens a page with a table containing all performed analyses for the given file (See Figure C.1). For a newly uploaded file, the table is empty. Clicking the Analyze button creates a new file analysis.

7.2.1 Analysis Overview Views

Selecting a file analysis takes the user to the traffic participants page (See Figure 7.2). This is one of three file overview pages. It displays a table entry for each IP address and port combination present in the analyzed file. For each endpoint, aggregate information regarding its traffic is shown, such as the number of incoming and outgoing DNS messages, how many of the DNS queries could be matched to the known domains, and whether and how many messages over the Bitcoin protocol have been sent or received. Detailed views for the given participant can be opened by clicking their respective button in the rightmost *Actions* column.

The second available overview page is Known Services (See Figure 7.3). It shows a card for each known domain name that has been queried inside the captured traffic. For each domain, it shows who queried, a list of IP addresses the query resulted in, and finally, a list of endpoints that accessed at least one of the resolved addresses.

The third available overview page shows all Bitcoin traffic (See Figure C.2). It should be roughly equivalent to opening a packet capture file in Wireshark with the `bitcoin` filter enabled. For each message, the sender and recipient are shown along with the time and Bitcoin command. For some Bitcoin message types, a message contents summary is also shown. Finally, the Actions column contains a button that opens the Bitcoin messages' detailed view.

7.2.2 Traffic Participant Views

For each traffic participant in the network analysis (See Figure 7.2), three views are available in the implemented application. The DNS Overview page (See Figure C.3) shows a list of all DNS queries and responses in the analyzed traffic. In total, there are three tables on this page. the first one shows DNS query and response exchanges, which could be paired by their Transaction ID. the other two tables contain unpaired DNS queries and responses, respectively. For each paired DNS exchange, the most notable columns are Query Name, which contains the queried domain, and Query Type, which describes the type of DNS record that was requested, the Response Addresses column, which shows how the domain was resolved, and finally, if the queried domain was matched as a known domain, the Purpose column contains the purpose of the match domains.

For example, if the application is seeded using the gathered data. a query for the domain `btc1.trezor.io` is marked with the purpose *Blockbook*.

The Known Domains Summary view allows the user to put into perspective the queried domains for a given product. For example, in Figure C.4, we can see that the participants' traffic contained queries for domains that are known to be present in both Ledger Live and Trezor Suite traffic. Furthermore, we can see that in regard to Trezor Suite, there was only one match for a domain of unknown purpose from a total of the seeded twelve. In comparison, we can see that the traffic contained queries for one of three analytics domains for Ledger Live, as well as a query for the only font domain, and queries for 12 of the 13 seeded domains with unknown purpose. Based on this view, the user can infer that the traffic participant was likely using the Ledger Live cryptocurrency wallet rather than Trezor Suite.

The Bitcoin Overview page shows the participants' Bitcoin protocol traffic. By default, the page splits the traffic into two tables. One containing only incoming Bitcoin messages and the second one only outgoing (See Figure C.5). Using the toggle button at the top of the page, these two tables can be merged for a more sequential view of the traffic (See Figure 7.4). For each Bitcoin message, the timestamp, command, remote party IP address, and port are shown, as well as a summary for some of the supported message types. Finally, the Actions column contains a button that opens the Bitcoin messages' detailed view.

7.2.3 Bitcoin Message Detail Views

Bitcoin messages can be opened in a dedicated detail view. the application currently supports and shows a more detailed view for the following message types: headers, tx, getdata, inv, and notfound.

The detailed view for tx Bitcoin messages contains two tables containing lists of inputs and outputs inside the transmitted transaction (See Figure C.6).

For headers messages, the application shows a table listing individual block hashes, shows the details of the transmitted transactions with a complete list of inputs and outputs (See Figure C.6).

Because getdata, inv, and notfound messages all contain so-called inventory vectors, they share the same display logic within the application. Figure 7.5 shows a detailed view of a getdata message. the detailed page contains an inventory items table containing individual inventory vector items. An inventory item can be further opened using the button in the Actions column.

This opens the Inventory Overview for the given inventory item, showing all Bitcoin messages in which the inventory item has been referenced.

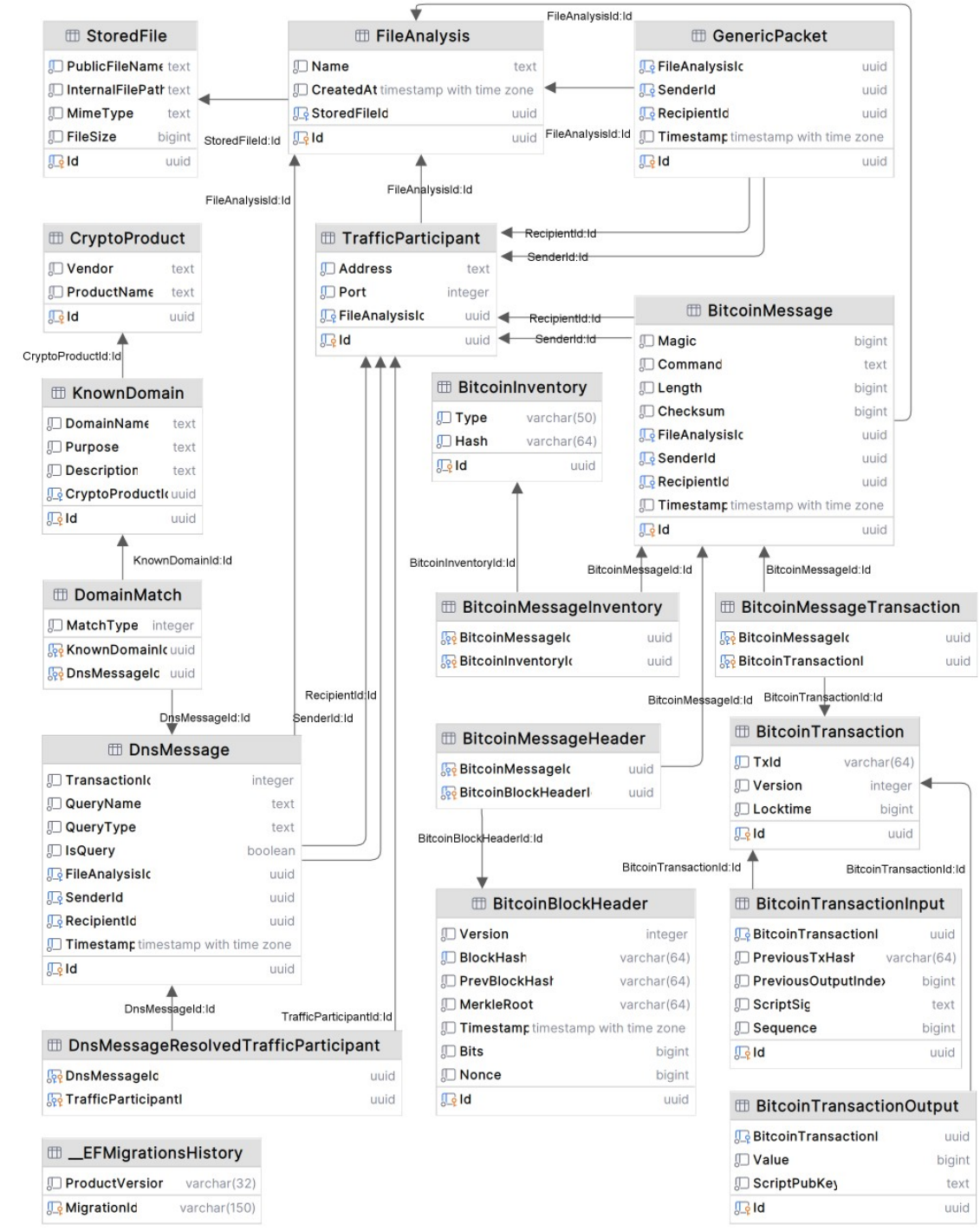


Figure 7.1: Database Schema

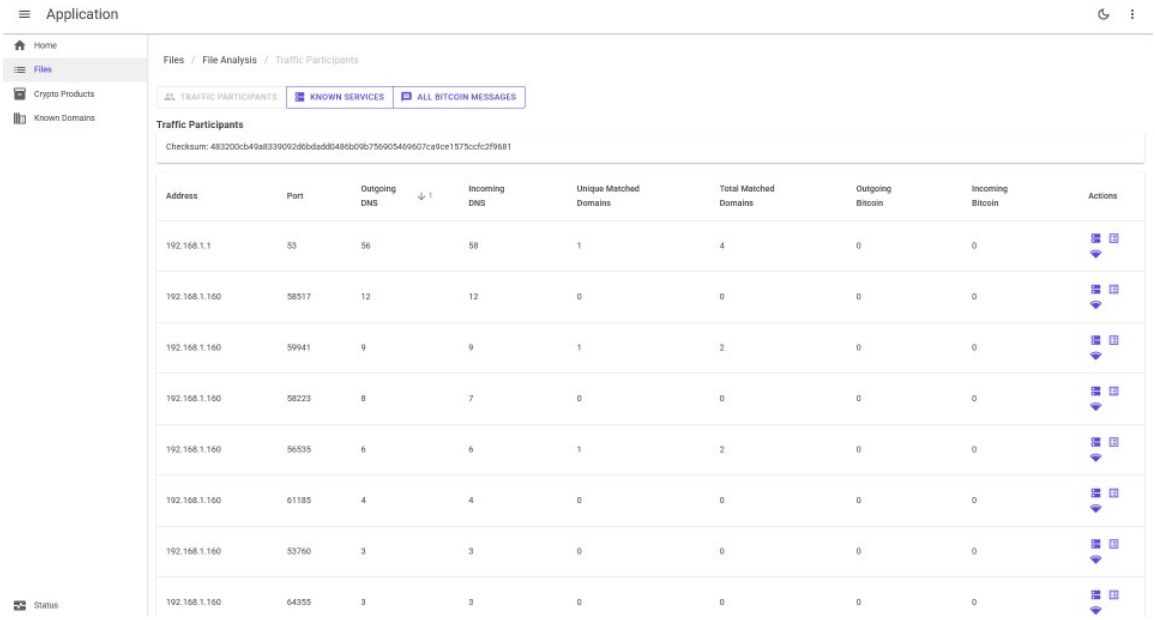


Figure 7.2: Application - Traffic Participants

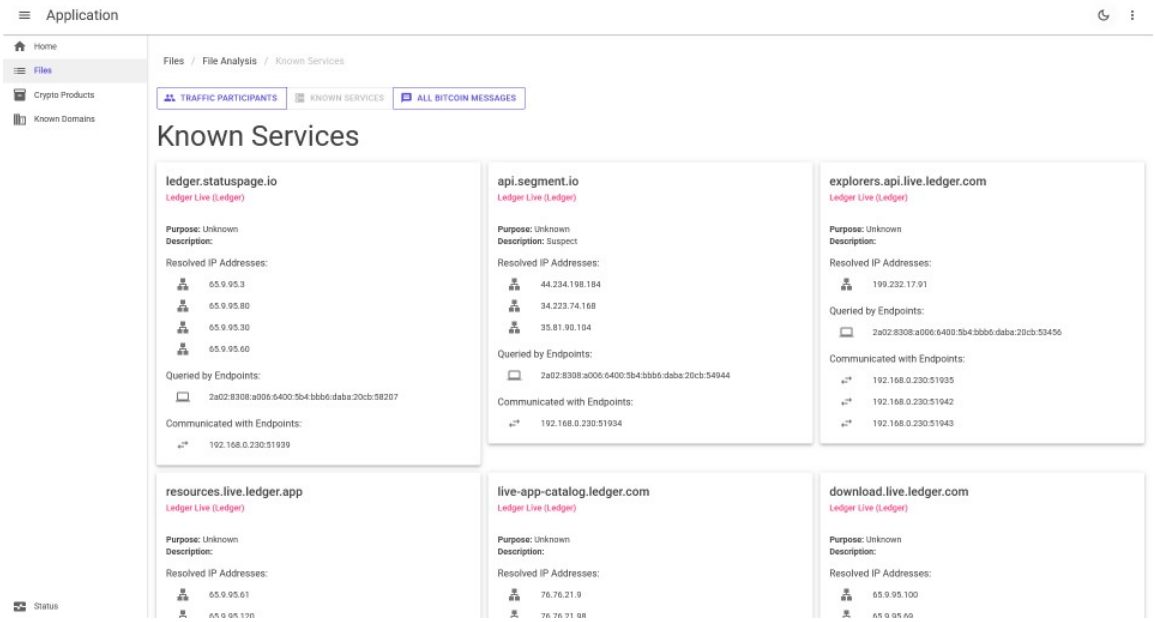


Figure 7.3: Application - Known Services

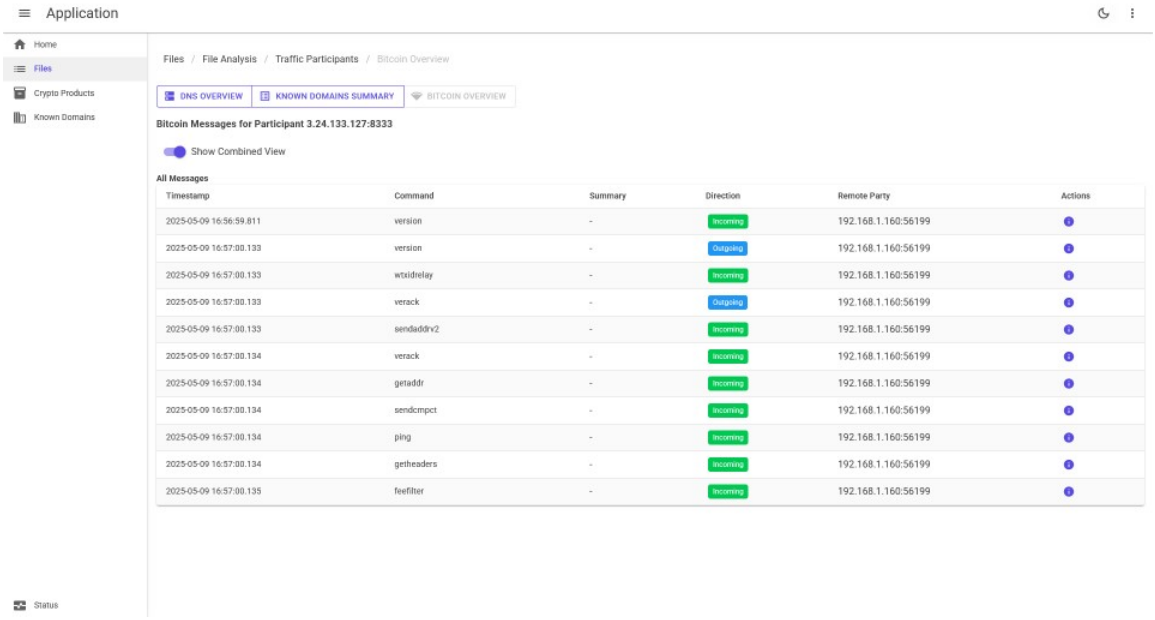


Figure 7.4: Application - Traffic Participant - Bitcoin Overview (Merged View)

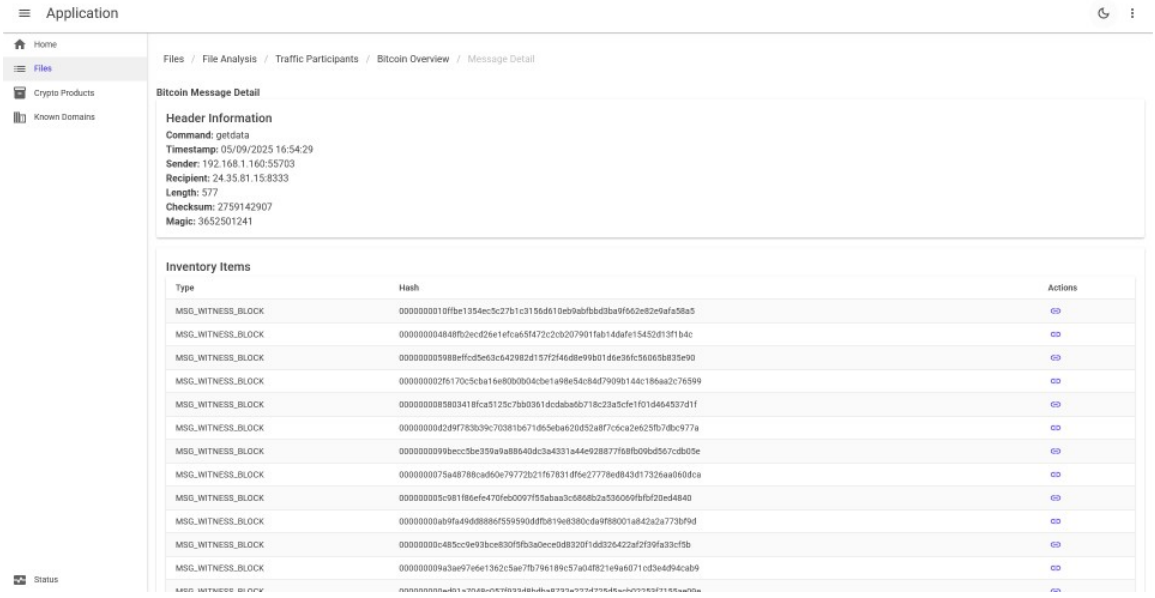


Figure 7.5: Application - Bitcoin Message Detail - getdata

Chapter 8

Testing

Application logic testing was performed using manually gathered input data. Packet capture files were collected across various scenarios. These packet captures were then spliced and combined for application logic testing scenarios. Packet capture files, both with and without cryptocurrency wallet traffic, were used during testing. For packet parsing and data display, the output of the implemented application was compared to the packet analyzer tool Wireshark. This chapter first focuses on all the data gathered for both research and testing of the application. It then focuses on test suites created for application logic testing of Domain Name Service and Bitcoin protocol-related features. And finally, it considers packet capture scenarios in which the application user would want to determine if a particular packet capture contains cryptocurrency-related traffic.

8.1 Testing Data Set

In the context of creating this work, packet captures were taken for both research and application testing. When using Trezor Suite, Ledger Live, and Electrum Wallet, network traffic captures were taken for the following scenarios:

- control test - packet capture contains only unrelated internet traffic;
- wallet website tests - cryptocurrency wallet is not running, packet capture contains regular traffic, and a visit to the website of the wallet vendor using an internet browser;
- open client tests - network traffic of launching the software wallet;
- transaction send - the user sends a transaction into the cryptocurrency network, capture starts after the initial wallet launch;
- balance view - the user views their balance, capture starts after the initial wallet launch;

A Bitcoin Core client (v28.1.0) was used to gather network traffic captures containing the Bitcoin protocol. Captures were taken in the following scenarios:

- initial block download (IBD)
  - start - begin the initialization process and start downloading data from the cryptocurrency network;

– resume - start a partially complete synchronization process;
– synchronizing - begin capture while the synchronization process is ongoing;
– start to end - start the synchronization for a period of time, and gracefully stop;

Using the larger packet captures containing tens of thousands of packets, smaller network data samples were created using Wireshark's *export specified packets* feature. By creating smaller, humanly graspable files, it was possible to manually compare data shown by the implemented application with data displayed by Wireshark. These slices were used for application logic testing (see Sections 8.2 and 8.3), and the scenarios in their entirety were used when evaluating the utility of the application for detection of cryptocurrency-related traffic inside a packet capture file (see Section 8.4).

8.2 Domain Name Service Analysis Testing

To test the application's capability to interpret DNS queries, a set of test cases was created. This test suite largely utilizes network traffic files containing traffic generated by Trezor Suite, Ledger Live, and Electrum Wallet, as well as captures taken as control tests, containing web browser traffic. See Figure 8.1 for a summary of the performed test cases. Each test case is described by a short name, goal, or description, and the name of the packet capture file in the appendices. the result column represents whether it was possible to accomplish the test's goals.

The purpose of the Single a Query test case is to simply verify basic DNS parsing functionality. the test case is considered a success if the application can show a DNS transaction.

The purpose of the Trezor Website test case is to present the application with a more realistic scenario. It contains unfiltered network traffic of a user accessing the Trezor website. the test case is successful if the application, seeded with known domains, can detect and show that traffic to a known domain was present.

The Ledger Live test case contains the launch of a software wallet, which is accompanied by several DNS queries to known domains. the test is considered successful if the application can show most known domain associations with the Ledger Live wallet.

8.3 Bitcoin Protocol Analysis Testing

Using the larger packet captures containing Bitcoin Core Client traffic, smaller network data samples were created using Wireshark's *export specified packets* feature. By creating smaller, humanly graspable files, it was possible to manually compare data shown by the implemented application with data displayed by Wireshark. See Figure 8.2 for a summary of the performed test cases. Each test case is described by a short name, goal, or description, and the name of the packet capture file in the appendices. the result

Test Case Name	Description/Goal	Result	File Name (*.pcapng)
Single a Query	known domain DNS transaction	Passed	dns-fonts_A
Trezor Website	traffic with known domain	Passed	trezor_website_test
Ledger Live	software wallet launch	Passed	2.73.1_LedgerNanoX

Table 8.1: DNS Protocol Test Cases

column represents whether the application results matched the reference output of Wireshark. The created test suite starts with a simple test case *Version*, which contains only a single multi-packet Bitcoin *version* message. the *Single Peer* test case goes a step further, with multiple (19) Bitcoin messages of varying types (*version, verack, wtxidrelay, sendaddrv2, sendcmpct, ping, pong, getheaders, feefilter*). the *Mixed Single* test case additionally also contains TCP traffic from different IP addresses, but Bitcoin protocol messages are still only from and to a single endpoint. the *Two Peers* test case contains Bitcoin protocol traffic from two different peers. the *Mixed multi* test case is then an extension, also containing other TCP communication in between on the same port. Finally, the *2.5k Packets* test case took a larger, unfiltered slice of traffic taken over 17 seconds, containing 192 Bitcoin messages.

8.3.1 Results

Analysis of all the above-listed files resulted in expected behavior. Meeting the set-out per-test case criteria, for DNS test cases, and matching the results of Wireshark with the *bitcoin* display filter for Bitcoin test cases. Screenshots of test results are attached as files. (See Appendix E.)

8.4 Software Wallet Related Traffic Detection

Individual files from the *packet capture* directory (Attachment directory structure can be seen in Appendix D) with the goal to determine whether the implemented application can be used to identify if the captured traffic contains an active cryptocurrency software wallet, and, if possible, the goal is to identify which software wallet generated the traffic.

8.4.1 Results

The application does not provide a yes or no answer to the presence of a cryptocurrency software wallet, but rather presents the user with a summary of the suspected traffic.

The implemented application marks services used by software wallets, even in some samples of regular Internet traffic. This is expected. For example, authentication services such as `accounts.google.com` are not exclusively used by Trezor Suite and Ledger Live. the access to these services is marked by the application as 'Authentication' to give the user a better picture of what the application detected. These false positives are also expected from visits to the wallet vendor's websites. This false positive was not encountered on a visit to the Electrum Wallet website because, unlike Ledger Live and Trezor Suite, Electrum

Test Case Name	Description/Goal	Result	File Name (*.pcapng)
Version	fragmented message	Match	bitcoin-version
Singe Peer	multiple messages from one peer	Match	bitcoin-one_peer
Mixed Single	contains other TCP traffic	Match	bitcoin-mixed_single
Two Peers	two bitcoin peers sequentially	Match	bitcoin-two_peers
Mixed Multi	multiple bitcoin streams	Match	bitcoin-mixed_multi
2.5k Packets	larger, unfiltered traffic slice	Match	bitcoin-2,5k

Table 8.2: Bitcoin Protocol Test Cases

Wallet uses services from third parties and does not host any service under their website domain.

For all surveyed lightweight software wallets, the implemented application detected connections to more than eight known services for the given wallet during the application launch. Among the detected services, there was always at least one cryptocurrency node. This was considered to be conclusive evidence of wallet traffic from the given vendor.

For Bitcoin Core, Bitcoin protocol traffic was present in all traffic samples. Even though some peers are communicating over the encrypted version of the peer-to-peer protocol, which the application can not read. Some peers are still running the unencrypted version of the protocol, which is detected and correctly parsed by the application.

In test cases where captured traffic does not contain the launch of a lightweight software wallet, the implemented application did not detect connections to enough known wallet services to determine whether a cryptocurrency software wallet is running.

Trezor Suite integrates an optional Tor client. When enabled, the implemented application did not detect enough data to determine whether a cryptocurrency wallet is present in the captured traffic. However, in a traffic sample where the Tor was toggled from on to off, the application detected several connections to services connected to Trezor Suite, which could be considered as conclusive evidence of Trezor Suite wallet traffic.

In the test cases presented, the implemented application did not generate unexpected false positives. When cryptocurrency wallet traffic was detected, it was always possible to identify the vendor of the said wallet. However, it was not possible to determine whether a cryptocurrency wallet is running from traffic samples taken once the application was already running. This is most likely due to the fact that the wallets establish all their connections during their launch. Finally, in traffic samples of Trezor Suite with an enabled Tor client, the application did not detect enough data to determine whether a cryptocurrency wallet is running. It should be noted that whenever a cryptocurrency wallet was detected, the source IP address and port of the client were always known.

Chapter 9

Conclusion

The work covered the basics of cryptocurrencies, taking Bitcoin as its focal point. It quickly showed the contrast of Ethereum's different architecture and introduced noteworthy features of Dash. Then it moved on to the past and present on-chain anonymity concerns that Bitcoin has faced in the past and is facing at the time of writing. It took time to fully understand not only the solutions to these issues, but also the underlying concepts and data structures in the Bitcoin protocol itself. Then it covered off-chain anonymity: key storage and how users interact with cryptocurrencies using different types of wallets. the work described the purpose, advantages, disadvantages, and limitations of existing wallets. It described considerations that must be made in the implementation of a cryptocurrency software wallet. the work then focuses on the traffic that is unavoidably generated by a cryptocurrency wallet when connecting to a cryptocurrency network. Then it selected three software wallets and analyzed their behavior in relation to the expectations mentioned above. Finally, once all the fundamentals of cryptocurrencies were sufficiently explained, several approaches to attack anonymity in cryptocurrencies were proposed, and one of them was chosen. Based on this research, an attack vector in cryptocurrency network traffic was proposed. User stories were conceptualized as use cases for the proposed application. the user stories, along with the gained theoretical domain knowledge from the research phase, lead to an application design proposal. Based on this design, an application was developed that can help its users determine whether a given traffic capture contains communication of a cryptocurrency wallet and, if so, try to identify its vendor. the testing of the implemented application showed that, in select cases, this is indeed possible, mainly when the launch of the cryptocurrency wallet was present in the captured traffic. The implemented application is published on GitHub[1].

9.1 Future Work

In regard to user usability, it might be beneficial to implement a way of merging what the application considers as 'Traffic Participants'. Right now, every IP and port combination is naively interpreted as a different user, which is rarely the case in real internet traffic. For example, when capturing network traffic on a computer's interface, the 'Participant' could be all ports for multiple IP addresses.

[1] https://github.com/DenisHromada/CryTraCtor

Because of the application's focus on relationships between individual entities, it might also be beneficial to use a graph NoSQL database, instead of a traditional database like the one used here.

Right now, the application implements only five Bitcoin message types. a natural extension of the application would implement all Bitcoin message types. Furthermore, Bitcoin protocol traffic over the encrypted version of the protocol (BIP 324) is currently ignored altogether, as the Bitcoin message parser looks for the Bitcoin mainnet magic. An extension of the application could also contain some indication that the encrypted traffic was present.

The traffic transferred by lightweight wallets seems to be mostly over encrypted TLS/SSL channels. An extension of the application could also access TLS certificates or otherwise take this traffic into account.

While the implemented application can be used to determine whether a traffic capture contains cryptocurrency wallet communication, it only presents the user with data, but not a verdict. As such, the user has to be familiar with cryptocurrency concepts laid out in this work to make the final judgment. This could be improved upon if the application also offered a probability score with signs of individual detected cryptocurrency wallets.

The implemented application could also be expanded with a dynamic database of known active cryptocurrency services. For example, by actively monitoring nodes in the cryptocurrency network, it could label traffic even for wallets configured with a non-standard cryptocurrency node. Likewise, since the shortcoming of the implemented application lies in packet captures that do not contain the launch of the analyzed wallets, the application could simulate this traffic. For example, it generates its own domain name queries and matches the resulting IP addresses with IP addresses contained in the captured traffic.

Bibliography

[1] Aiolli, F.; Conti, M.; Gangwal, A. and Polato, M. *Mind Your Wallet's Privacy: Identifying Bitcoin Wallet Apps and User's Actions through Network Traffic Analysis.* New York, NY, USA: Association for Computing Machinery, 2019. Available at: https://doi.org/10.1145/3297280.3297430.

[2] bitcoincoreorg. *Bitcoin Core* online. Available at: https://bitcoincore.org/. [cit. 2025-05-01].

[3] Buterin, V. *A next-generation smart contract and decentralized application platform* online. 2014. Available at: https://ethereum.org/en/whitepaper/.

[4] De Lucia, M. J. and Cotton, C. Identifying and detecting applications within TLS traffic. In: SPIE. *Cyber Sensing 2018.* 2018, vol. 10630, p. 179–190.

[5] Deuber, D. and Schröder, D. CoinJoin in the Wild. In: Bertino, E.; Shulman, H. and Waidner, M., ed. *Computer Security – ESORICS 2021.* Cham: Springer International Publishing, 2021, p. 461–480. ISBN 978-3-030-88428-4.

[6] Docker Inc.. *Docker* online. Available at: https://www.docker.com/. [cit. 2024-01-14].

[7] Duffield, E. and Diaz, D. *Dash: A privacycentric cryptocurrency* online. 2015. Available at: https://www.exodus.com/assets/docs/dash-whitepaper.pdf. [cit. 2024-03-03].

[8] Ficsor, A. WasabiVsSamourai. *GitHub* online. Available at: https://github.com/nopara73/WasabiVsSamourai. [cit. 2024-01-10].

[9] Ficsor, A.; Kogman, Y.; Ontivero, L. and András Seres, I. *WabiSabi: Centrally Coordinated CoinJoins with Variable Amounts* Cryptology ePrint Archive, Paper 2021/206. 2021. Available at: https://eprint.iacr.org/2021/206.

[10] Ficsór Ádám and Hill, W. L. *ZeroLink: The Bitcoin Fungibility Framework.* 2020. Available at: https://github.com/nopara73/ZeroLink.

[11] Gervais, A.; Capkun, S.; Karame, G. O. and Gruber, D. On the privacy provisions of bloom filters in lightweight bitcoin clients. In: *Proceedings of the 30th Annual Computer Security Applications Conference.* 2014, p. 326–335.

[12] Honzik, T. Bitcoin address types compared: P2PKH, P2SH, P2WPKH, and more. *Unchained* online. Available at: https://unchained.com/blog/bitcoin-address-types-compared/. [cit. 2024-01-10].

[13] Hromada, D. *Forensic Analysis of Anonymization Principles of Cryptocurrency Networks* online. 2024 [cit. 2025-05-14]. Available at: https://theses.cz/id/c6laqc/.

[14] IBM. *What is three-tier architecture?* online. Available at: https://www.ibm.com/think/topics/three-tier-architecture. [cit. 2025-05-01].

[15] Kaitai Project. *Kaitai Struct* online. Available at: https://kaitai.io/. [cit. 2024-04-30].

[16] Ledger Academy. *What is a Software Wallet?* online. June 2023. Available at: https://www.ledger.com/academy/topics/security/what-is-a-software-wallet. [cit. 2024-01-10].

[17] Meiklejohn, S.; Pomarole, M.; Jordan, G.; Levchenko, K.; McCoy, D. et al. A fistful of bitcoins: characterizing payments among men with no names. In: *Proceedings of the 2013 conference on Internet measurement conference*. 2013, p. 127–140.

[18] Microsoft Corporation. *Entity Framework Core* online. Available at: https://learn.microsoft.com/en-us/ef/core/. [cit. 2024-04-30].

[19] Microsoft Corporation. *Microsoft Learn* online. Available at: https://learn.microsoft.com/en-us/. [cit. 2025-04-30].

[20] Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System. *Decentralized business review* online, october 2008. Available at: http://www.bitcoin.org/bitcoin.pdf. [cit. 2023-04-25].

[21] Nakamoto, S. *Bitcoin v0.1 released* online. 8. January 2009. Available at: https://www.metzdowd.com/pipermail/cryptography/2009-January/014994.html. [cit. 2025-01-10]. The download link no longer leads to the v0.1 client zip.

[22] Popper, N. Mt. Gox creditors seek trillions where there are only millions. *The New York Times* online. The New York Times, May 2016. Available at: https://www.nytimes.com/2016/05/26/business/dealbook/mt-gox-creditors-seek-trillions-where-there-are-only-millions.html. [cit. 2023-11-07].

[23] Rooney, K. FTX customers who lost a fortune on the bankrupt exchange are doubling down on crypto. *CNBC*. CNBC, Oct 2023. Available at: https://www.cnbc.com/2023/10/02/ftx-customers-who-lost-fortune-are-doubling-down-on-crypto-.html.

[24] SatoshiLabs. *Transitioning from coinjoin in Trezor Suite* online. Available at: https://trezor.io/learn/a/coinjoin-in-trezor-suite. [cit. 2024-05-01].

[25] SatoshiLabs. *What is a hardware wallet how does it work?* online. Available at: https://trezor.io/learn/a/what-is-a-hardware-wallet. [cit. 2025-01-11].

[26] SatoshiLabs. *Important Update: Transitioning from CoinJoin in Trezor Suite* online. May 2024. Available at: https://blog.trezor.io/important-update-transitioning-from-coinjoin-in-trezor-suite-9dfc63d2662f. [cit. 2025-01-12].

[27] Schneider, N. *Recovering Bitcoin private keys using weak signatures from the blockchain* online. Available at: https://web.archive.org/web/20160308014317/http://www.nilsschneider.net/2013/01/28/recovering-bitcoin-private-keys.html. [cit. 2013-01-28].

[28] Sengupta, S.; Ganguly, N.; De, P. and Chakraborty, S. Exploiting Diversity in Android TLS Implementations for Mobile App Traffic Classification. In: *The World Wide Web Conference*. New York, NY, USA: Association for Computing Machinery, 2019, p. 1657–1668. WWW '19. ISBN 9781450366748. Available at: https://doi.org/10.1145/3308558.3313738.

[29] Statista Inc.. *Most popular relational DBMS* online. 2024. Available at: https://www.statista.com/statistics/1131568/worldwide-popularity-ranking-relational-database-management-systems/. [cit. 2024-04-30].

[30] Statista Inc.. *Biggest cryptocurrency in the world* online. 2025. Available at: https://www.statista.com/statistics/1269013/biggest-crypto-per-category-worldwide/. [cit. 2025-01-11].

[31] Statista Inc.. *Number of cryptocurrencies 2013-2025* online. 2025. Available at: https://www.statista.com/statistics/863917/number-crypto-coins-tokens/. [cit. 2025-01-11].

[32] Streetside Development LLC.. *Samourai Documentation* online. Available at: https://docs.samourai.io/. [cit. 2024-03-03].

[33] The PostgreSQL Global Development Group. *PostgreSQL* online. Available at: https://www.postgresql.org/. [cit. 2024-04-30].

[34] United States Department of Justice. *Founders And CEO Of Cryptocurrency Mixing Service Arrested And Charged With Money Laundering And Unlicensed Money Transmitting Offenses*. April 2024. Available at: https://www.justice.gov/usao-sdny/pr/founders-and-ceo-cryptocurrency-mixing-service-arrested-and-charged-money-laundering.

[35] Wuille, P. Hierarchical Deterministic Wallets. *GitHub* online. Available at: https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki. [cit. 2024-02-27].

[36] Wuille, P. Version 2 P2P Encrypted Transport Protocol. *GitHub* online. Available at: https://github.com/bitcoin/bips/blob/master/bip-0324.mediawiki. [cit. 2025-02-27].

[37] Yang, J.; Sun, G.; Xiao, R.; He, H. et al. Detectable, traceable, and manageable blockchain technologies BHE: an attack scheme against bitcoin p2p network. *Wireless Communications and Mobile Computing*. Hindawi, 2022, vol. 2022.

[38] Zorinaq. *Bitcoin v0.1 Source Code* online. 10. March 2012. Available at: https://bitcointalk.org/index.php?topic=68121.0. [cit. 2025-01-10]. Http://www.zorinaq.com/pub/bitcoin-0.1.0.rar.

Appendix A

Bitcoin Data Structures

Each block on the blockchain consists of a block header and transactions. Every transaction in a block has the following fields[1]:

Name	Description
`Version`	Transaction version number
`Flag`	Indicates use of Segregated Witness
`tx_in count`	Number of transaction inputs
`tx_in`	Transaction inputs
`tx_out count`	Number of transaction outputs
`tx_out`	Transaction outputs
`Witnesses`	A list of witnesses only present if Flag indicates use of Segregated Witness
`lock_time`	If non-zero, prevents transaction from being spent before a specified block height or time

Table A.1: Fields in a Bitcoin block

Name	Description
Outpoint hash	Hash of transaction containing the spendable output
Outpoint index	The index within the previous transaction of the spendable output
Script length	Length of script signature
Script signature (ScriptSig)	Inputs to satisfy (pubkey script) the spending of the output
Sequence number	Used to enable replace-by-fee if `<0xFFFFFFFF`

Table A.2: Fields in a Bitcoin input

Note, Bitcoin transactions do not have a 'destination' per se. Transaction outputs have a `sigScript` field, which is a predicate that, given the right inputs, is true and the output is spendable in the given transaction. The given examples simplify this mechanism as a 'destination', where being able to fulfill this predicate makes a user the recipient.

[1] https://en.bitcoin.it/wiki

Name	Description
Value	Number of satoshis assigned to this output. (100 000 000 satoshi = 1 BTC)
Script length	Length of (scriptPubkey) script
Script (scriptPubKey)	Sets conditions that have to be fulfilled for the output to be spent.

Table A.3: Fields in a Bitcoin output

Appendix B

Cryptocurrency Node Seeds

The implemented application uses manually collected domain names of services used by cryptocurrency software wallets. This data was collected for wallets Ledger Live, Trezor Suite and Electrum Wallet and stored into separated comma separated value (csv) files. These files are attached with this work in the `Collected Domain Names` folder. Below is an example of data collected for Trezor Suite.

```
Vendor,ProductName,DomainName,Purpose,Description
Trezor,Trezor Suite,trezor.io,Vendor Root,
Trezor,Trezor Suite,o117836.ingest.sentry.io,Analyrics,
Trezor,Trezor Suite,data.trezor.io,Analyrics,
Trezor,Trezor Suite,wiki.trezor.io,Website,
Trezor,Trezor Suite,suite.trezor.io,Website,
Trezor,Trezor Suite,shop.trezor.io,Website,
Trezor,Trezor Suite,status.trezor.io,Website,Status Page
Trezor,Trezor Suite,btc1.trezor.io,Blockbook,Bitcoin
Trezor,Trezor Suite,btc2.trezor.io,Blockbook,Bitcoin
Trezor,Trezor Suite,btc3.trezor.io,Blockbook,Bitcoin
Trezor,Trezor Suite,btc4.trezor.io,Blockbook,Bitcoin
Trezor,Trezor Suite,btc5.trezor.io,Blockbook,Bitcoin
Trezor,Trezor Suite,bch1.trezor.io,Blockbook,Bitcoin Cash
Trezor,Trezor Suite,bch2.trezor.io,Blockbook,Bitcoin Cash
Trezor,Trezor Suite,bch3.trezor.io,Blockbook,Bitcoin Cash
Trezor,Trezor Suite,bch4.trezor.io,Blockbook,Bitcoin Cash
Trezor,Trezor Suite,bch5.trezor.io,Blockbook,Bitcoin Cash
Trezor,Trezor Suite,dgb1.trezor.io,Blockbook,DigiByte
Trezor,Trezor Suite,dgb2.trezor.io,Blockbook,DigiByte
Trezor,Trezor Suite,zec1.trezor.io,Blockbook,Zcash
Trezor,Trezor Suite,zec2.trezor.io,Blockbook,Zcash
Trezor,Trezor Suite,zec3.trezor.io,Blockbook,Zcash
Trezor,Trezor Suite,zec4.trezor.io,Blockbook,Zcash
Trezor,Trezor Suite,zec5.trezor.io,Blockbook,Zcash
Trezor,Trezor Suite,ltc1.trezor.io,Blockbook,Litecoin
Trezor,Trezor Suite,ltc2.trezor.io,Blockbook,Litecoin
Trezor,Trezor Suite,ltc3.trezor.io,Blockbook,Litecoin
...truncated for brevity
```

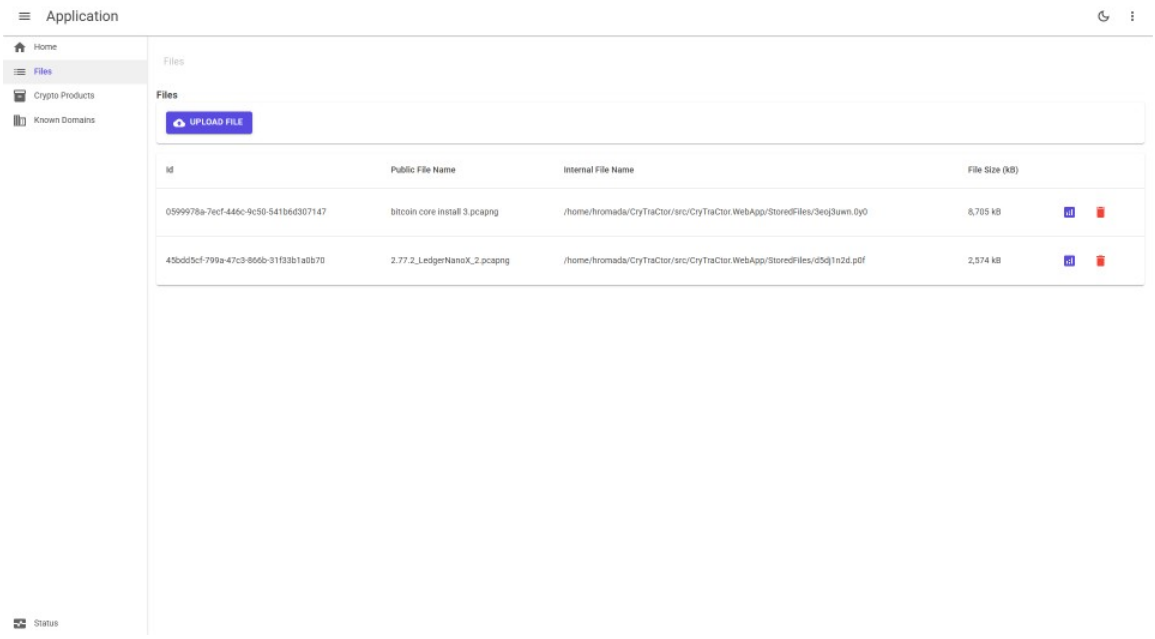


Figure B.1: Application - Files Page

Appendix C

Application Screenshots

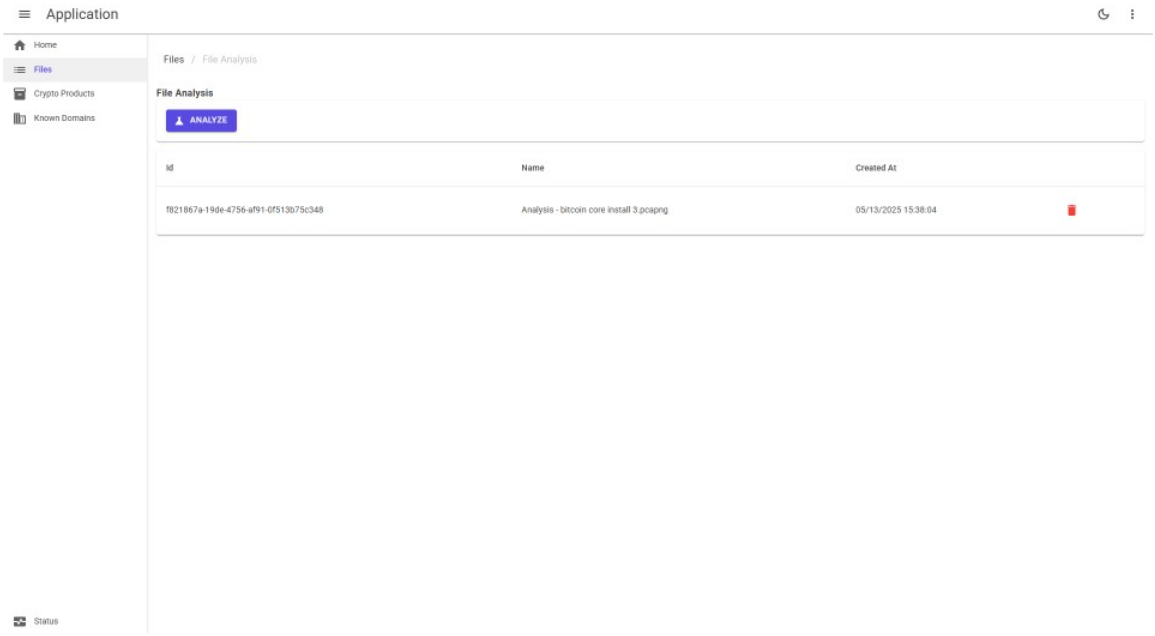


Figure C.1: Application - File Analysis

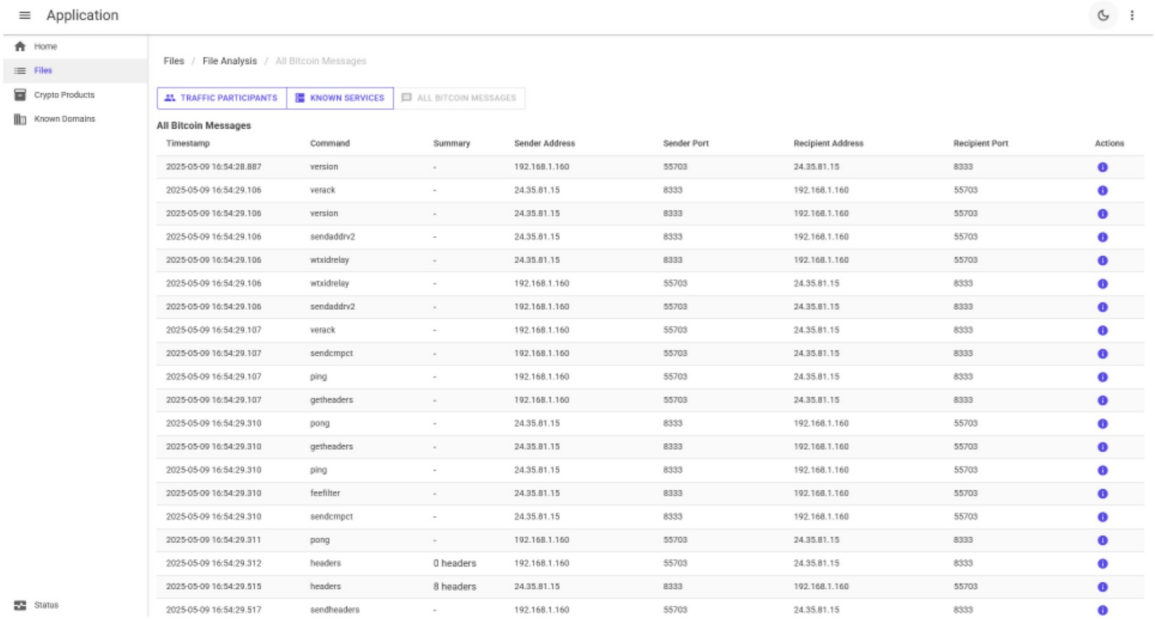


Figure C.2: Application - All Bitcoin Messages

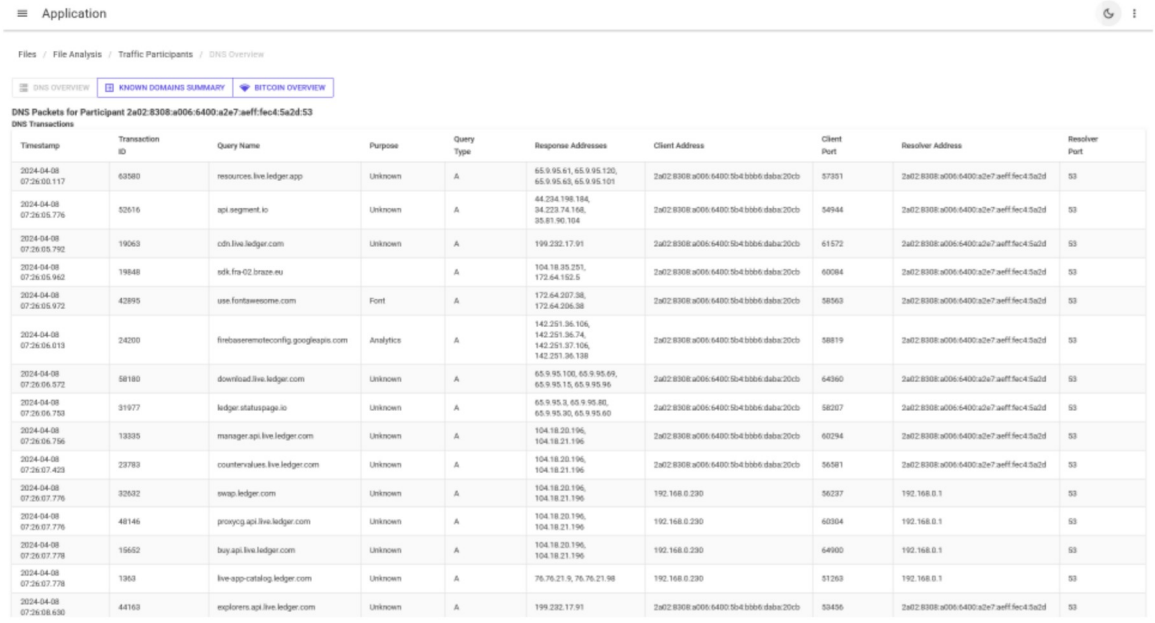


Figure C.3: Application - Traffic Participant - DNS Overview

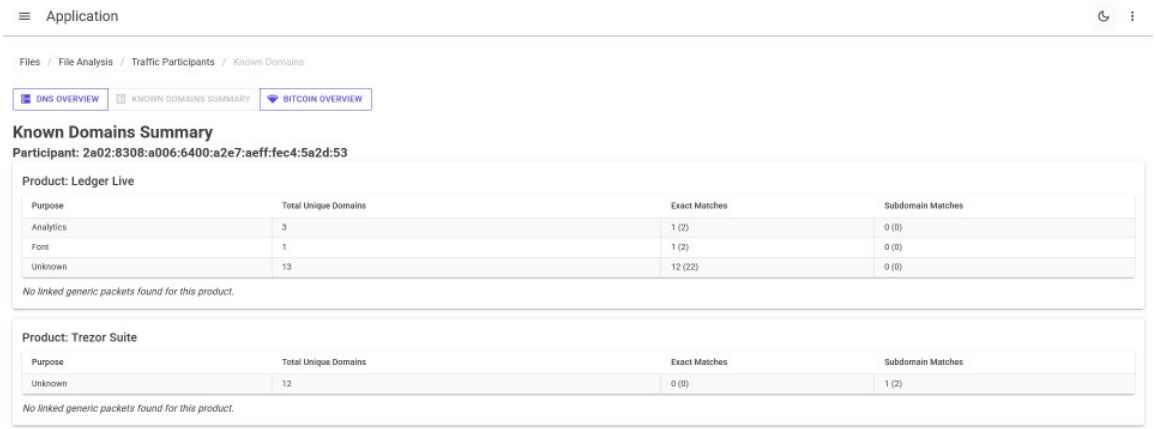


Figure C.4: Application - Traffic Participant - Known Domains Summary

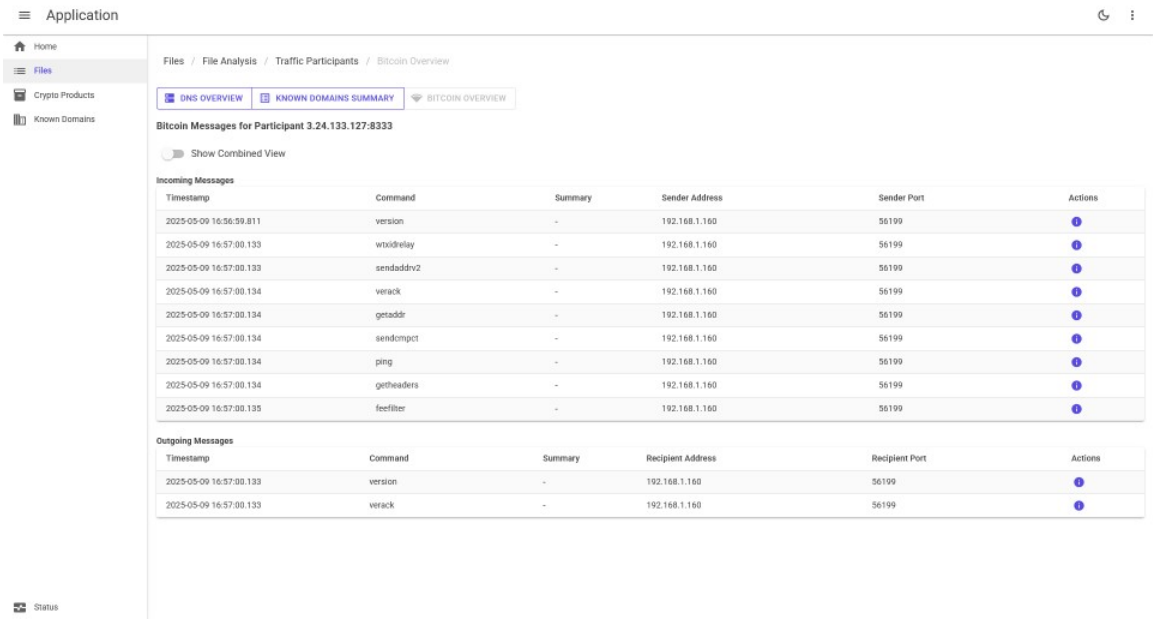


Figure C.5: Application - Traffic Participant - Bitcoin Overview (Split View)

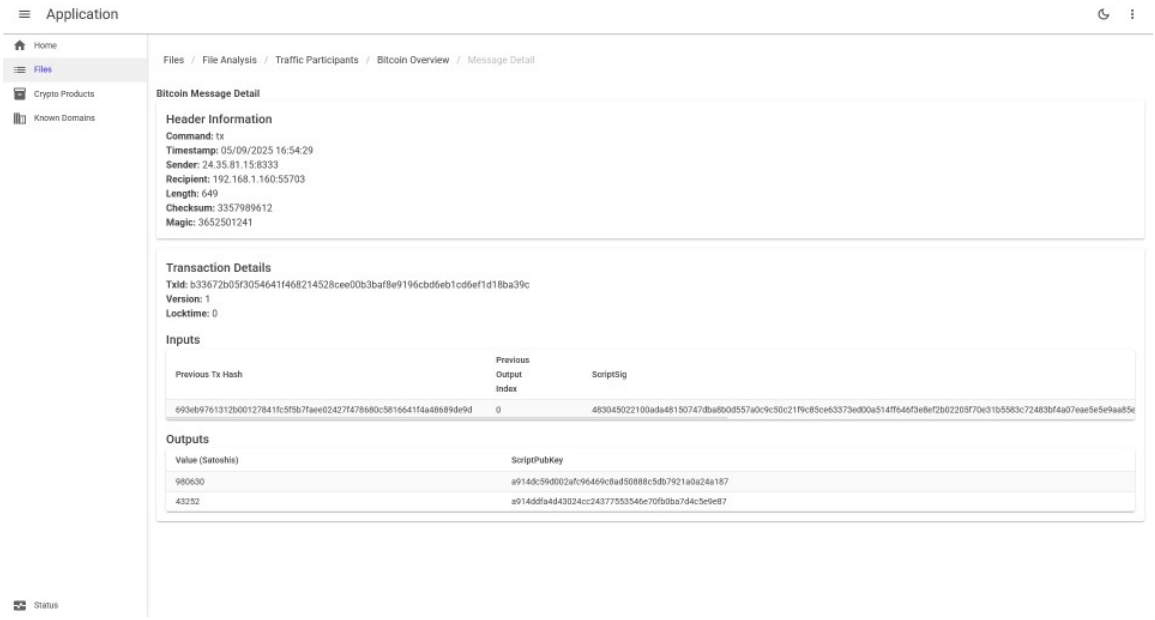


Figure C.6: Application - Bitcoin Message Detail - tx

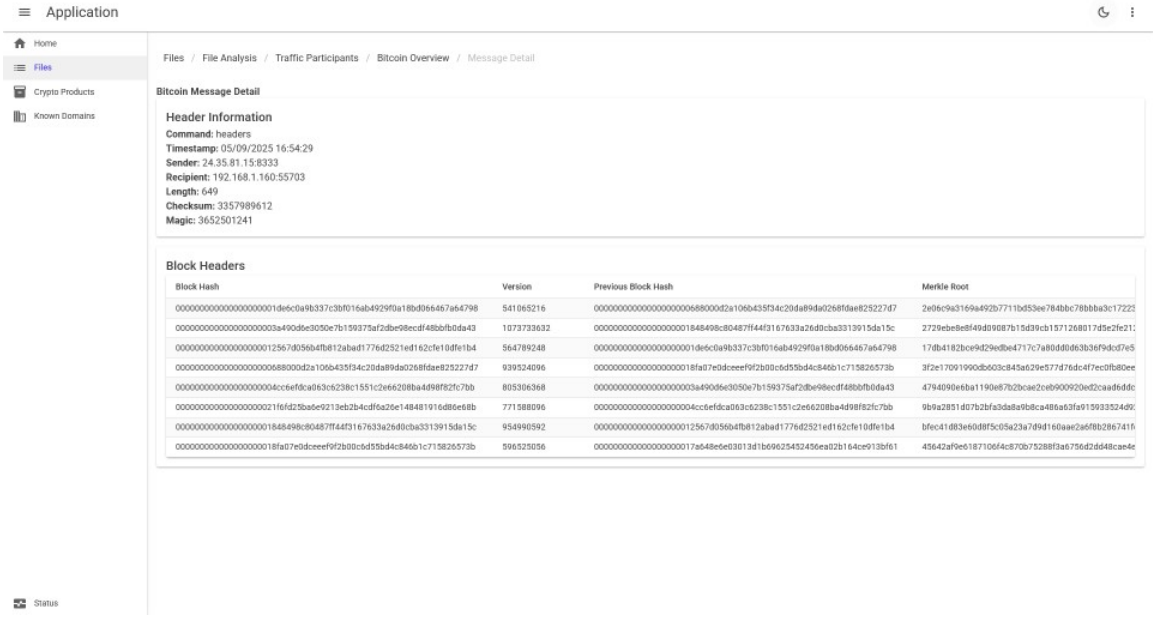


Figure C.7: Application - Bitcoin Message Detail - headers

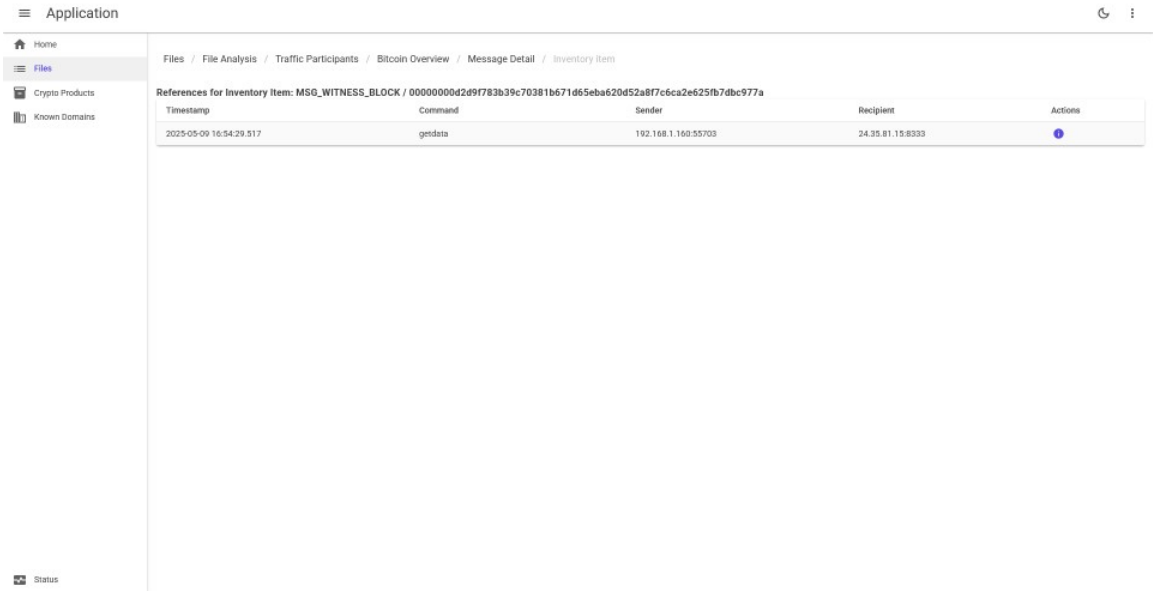


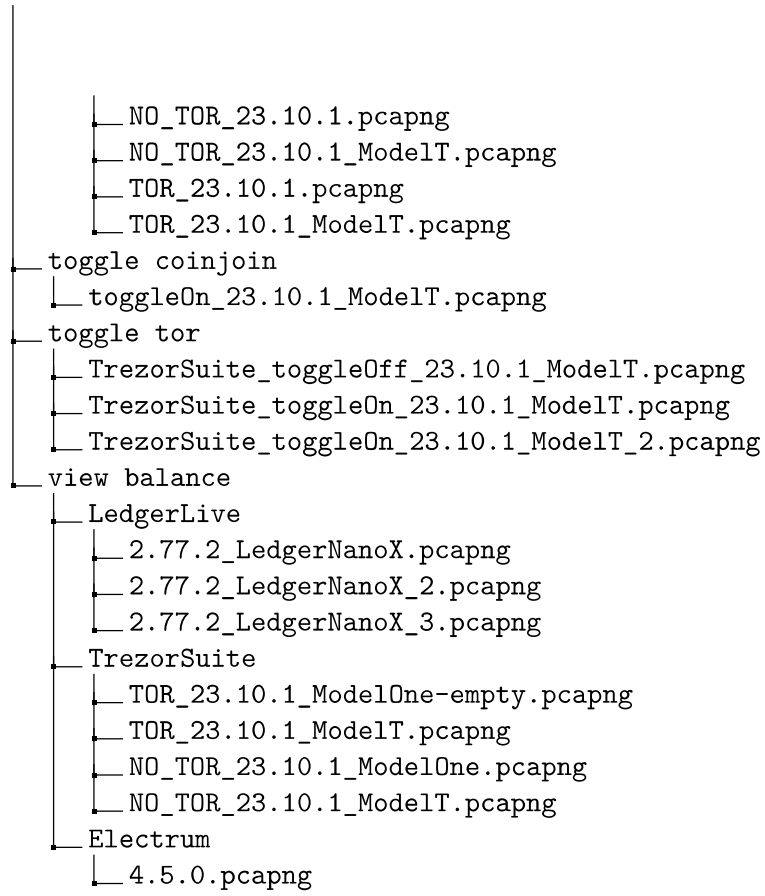
Figure C.8: Application - Bitcoin Inventory Item Overview

Appendix D

Cryptocurrency Software Wallet Packet Captures

For the purposes of this work, traffic of software wallets Ledger Live, Trezor Suite and Electrum Wallet was captured. These captures are attached in the `.pcap` format in the `Collected Packet Captures` folder.

```
Collected Packet Captures
├── bitcoin
│   ├── bitcoin core install 1 begin.pcapng
│   ├── bitcoin core install 2 end.pcapng
│   ├── bitcoin core install 3 begin end.pcapng
│   └── bitcoin core install 4 middle.pcapng
├── testing
│   ├── control_test.pcapng
│   ├── electrum_website_test.pcapng
│   ├── github_test.pcapng
│   ├── ledger_website_test.pcapng
│   └── trezor_website_test.pcapng
└── research
    ├── open client
    │   ├── LedgerLive
    │   │   ├── 2.73.1_LedgerNanoX.pcapng
    │   │   ├── 2.77.2_LedgerNanoX.pcapng
    │   │   └── 2.77.2_LedgerNanoX_2.pcapng
    │   ├── TrezorSuite
    │   │   ├── NO_TOR_23.10.1_ModelOne.pcapng
    │   │   ├── NO_TOR_23.10.1_ModelT.pcapng
    │   │   ├── TOR_23.10.1_ModelOne.pcapng
    │   │   └── TOR_23.10.1_ModelT.pcapng
    │   └── Electrum
    │       ├── 4.5.0.pcapng
    │       └── 4.5.0_2.pcapng
    └── send-sent
        ├── 2.77.2_LedgerNanoX.pcapng
        └── TrezorSuite
```


NO_TOR_23.10.1.pcapng
NO_TOR_23.10.1_ModelT.pcapng
TOR_23.10.1.pcapng
TOR_23.10.1_ModelT.pcapng
toggle coinjoin
toggleOn_23.10.1_ModelT.pcapng
toggle tor
TrezorSuite_toggleOff_23.10.1_ModelT.pcapng
TrezorSuite_toggleOn_23.10.1_ModelT.pcapng
TrezorSuite_toggleOn_23.10.1_ModelT_2.pcapng
view balance
LedgerLive
2.77.2_LedgerNanoX.pcapng
2.77.2_LedgerNanoX_2.pcapng
2.77.2_LedgerNanoX_3.pcapng
TrezorSuite
TOR_23.10.1_ModelOne-empty.pcapng
TOR_23.10.1_ModelT.pcapng
NO_TOR_23.10.1_ModelOne.pcapng
NO_TOR_23.10.1_ModelT.pcapng
Electrum
4.5.0.pcapng

Appendix E

Test Results

The test results of each test case are present in the attachments in the `tests` directory:

```
tests
├── dns_a.png
├── dns_trezor_website.png
├── dns_ledger_live.png
├── bitcoin_version.png
├── bitcoin_one_peer.png
├── bitcoin_two_peers.png
├── bitcoin_mixed_single.png
├── bitcoin_mixed_multi.png
└── bitcoin_2,5k.png
```